

IFQM Employee Ideation Platform

Test Case and Assurance Document

Functionality - Safety - Reliability - Scalability - Data Integrity - Future Scope

Document	EIT_TestCases_Simple.pdf
System under test	IFQM Employee Ideation Platform (multi-tenant, role-based) - Node/Express API + React front end + MySQL
Test method	Automated runner driving the real application over HTTP; nothing mocked
Evidence source	backend/test/tc_results.json, written by backend/test/tc_runner.mjs
Run completed	2026-07-28 17:35:05
Total test cases	228
Passed	228 (100.0%)
Failed	0
Modules covered	22
Assurance dimensions	8
Document generated	2026-07-28 23:05

Every "Actual Output" in this document is a verbatim record of what the running system did during the run named above. Nothing in the results tables is written by hand, and a case that fails is printed as a failure rather than removed.

Contents

Contents	2
1. Purpose and scope	5
1.1 Audience, and how to use this document	5
1.2 How to read a test case	5
2. The system under test	5
2.1 Tenancy model	5
2.2 Roles	6
2.3 Idea lifecycle	6
2.4 The life of one request	6
3. Test environment and method	7
3.1 Environment	7
3.2 What "Actual Output" is allowed to be	7
3.3 Reproducing this run	7
3.4 The rules a case has to follow	8
3.5 Threat model behind the safety cases	8
4. Test approach, dimension by dimension	9
4.1 Functional behaviour	9
4.2 Safety, security and privacy	9
4.3 Reliability and fault tolerance	9
4.4 Scalability - horizontal	10
4.5 Scalability - vertical	10
4.6 Data integrity and recovery	10
4.7 Extensibility and future scope	10
4.8 Observability and operations	10
4.9 What this suite does not cover	10
5. Results	11
5.1 By assurance dimension	11
5.2 By module	11
6. Defects found by this cycle, and their fixes	13
6.1 Simultaneous submissions collided on the idea code	13
6.2 An idea could be approved five times over	13
6.3 Anonymous submissions were identifiable from the approval timeline	13
6.4 Observations that were not defects	14
7. Measured behaviour under load	15

7.1 Dataset	15
7.2 Latency at 5,000 ideas	15
7.3 Throughput, concurrency and footprint	15
7.4 Two instances behind one database	15
7.5 What the measurements imply for sizing	16
8. Coverage matrix	17
9. Residual risks, limitations and recommendations	18
9.1 Future scope, and what already supports it	18
10. Test cases in full	19
10.1 Authentication (12/12 passed)	19
10.2 Platform Admin (10/10 passed)	20
10.3 Org Admin / Users (6/6 passed)	20
10.4 Ideas (8/8 passed)	21
10.5 Voting & Community (6/6 passed)	21
10.6 Comments (5/5 passed)	22
10.7 Review & Approval (5/5 passed)	22
10.8 Categories & Challenges (4/4 passed)	23
10.9 Analytics & Reports (4/4 passed)	23
10.10 Notifications (3/3 passed)	24
10.11 Reports & Export (3/3 passed)	24
10.12 Branding & Settings (3/3 passed)	24
10.13 Support (3/3 passed)	24
10.14 QCMS Integration (7/7 passed)	25
10.15 Security & Multi-Tenancy (5/5 passed)	25
10.16 Safety & Data Protection (40/40 passed)	26
10.17 Reliability & Fault Tolerance (24/24 passed)	29
10.18 Scalability - Horizontal (16/16 passed)	31
10.19 Scalability - Vertical & Performance (21/21 passed)	32
10.20 Data Integrity & Recovery (17/17 passed)	34
10.21 Extensibility & Future Scope (18/18 passed)	35
10.22 Observability & Operations (8/8 passed)	37
11. Appendix A - Suite inventory	39
12. Appendix B - Endpoint inventory and access rules	40
13. Appendix C - Configuration reference	44
14. Appendix D - Test data and accounts	45
15. Appendix E - Requirement traceability	45

16. Appendix F - Glossary	46
17. Appendix G - The assertion suite	47
18. Conclusion	48

1. Purpose and scope

This document is the test record for the IFQM Employee Ideation Platform. It exists to answer four questions that an evaluating organisation asks before putting a system in front of its whole workforce: does it do what it claims, is it safe with people's data, does it stay up when things go wrong, and will it still work when the organisation is ten times larger than it is today. A fifth question - can this be extended without a rewrite - is answered in the same way, because a system that cannot absorb the next requirement becomes a liability whatever its test results say.

The scope is the whole product as deployed: the HTTP API that every client uses, the multi-tenant database layer beneath it, the authentication and authorisation rules, the file handling, the outbound integration with the QCMS implementation tool, and the operational surfaces (health probes, logging, migrations, backups) that keep it running. The React front end is exercised indirectly: every screen in it is a client of the endpoints tested here, and the rules that matter are enforced on the server precisely so that a browser cannot talk its way around them.

1.1 Audience, and how to use this document

It is written for four readers, and each can stop at a different point. An **evaluator or buyer** needs sections 5 to 9: the results, what the testing found, the measured behaviour under load, and the residual risks stated plainly. An **IT or security reviewer** will want section 3.4 (the threat model), section 4.2 (the safety approach), and then the Safety and Data Protection cases in section 10 in full. An **operations team** should read sections 7 and 9 with appendices B and C, which list every endpoint with its access rule and every configuration setting that a claim in this document depends on. A **developer joining the project** should read section 3 and then run the suite; it is the fastest accurate description of how the system behaves.

1.2 How to read a test case

Each case is one row with seven fields, the same format used across this programme:

- **Test Case ID** - stable identifier of the form TC-<MODULE>-<NNN>.
- **Module** - the functional or assurance area the case belongs to.
- **Functionality** - what was done, described as an action a person or a client performs.
- **Expected Output** - the behaviour the system is required to show. Written before the run, not after it.
- **Actual Output** - what the running system actually did, recorded automatically, including measured latency, row counts and error text where those are the point of the case.
- **Result** - Pass only when the actual output satisfies the expectation. Anything else is a Fail.
- **Timestamp** - when that case executed.

Negative cases are as important as positive ones and are written the same way. "User enters an email that does not exist" expects a refusal, so a refusal is a Pass and a successful login would be a Fail. Roughly half the suite is of this kind: it is easy to show that a feature works, and much more useful to show that it refuses to work when it should.

2. The system under test

The platform collects improvement ideas from employees, routes them through a review hierarchy, rewards participation with points and recognition, and hands approved ideas to the QCMS tool where implementation is tracked. It is sold to multiple organisations at once, which is the single most important fact about its architecture and about this test plan.

2.1 Tenancy model

Each customer organisation is a tenant with its own MySQL schema. A registry database holds the list of tenants and, per tenant, the database host and schema name. A request is bound to exactly one tenant by the organisation slug carried inside the caller's signed token; the connection pool handed to the request is that tenant's pool and no other. Cross-tenant leakage is therefore not prevented by a WHERE clause that somebody might forget - the other organisation's rows are in a different schema that the request never opens. Several cases in this document attack that boundary directly by presenting one tenant's record identifiers to another tenant's session.

2.2 Roles

Role	What it can do	Where the account lives
Platform Admin	Vendor staff. Creates and suspends organisations, manages support tickets and platform settings. Sees tenant counts and admin contacts only - never a tenant's ideas, users or files.	Registry database
Super Admin / Admin	Runs one organisation: users, hierarchy, approval chain, categories, challenges, branding, mail settings, analytics, exports, and the QCMS integration.	That tenant
Plant Head / Executive	Organisation-wide visibility of ideas and analytics; final approval authority.	That tenant
Senior Manager / Department Manager / Manager	Reviews ideas from their reporting line, approves, rejects or escalates, records ROI and implementation progress.	That tenant
Team Lead / Project Lead	First-level review of their team's ideas.	That tenant
Employee / Trainee	Submits ideas, attaches evidence, names co-suggesters, votes, comments, earns points.	That tenant

The role in the signed token is never trusted. On every single request the user row is re-read from the tenant database and the role, the account status and the password-change marker are taken from that row. Three cases in the safety module exist purely to prove this: a token whose role claim has been raised to admin, a token belonging to an account that has since been deactivated, and a token issued before a password change.

2.3 Idea lifecycle

Draft - Submitted - Under Review - Approved or Rejected - Implemented. Approval can escalate up the reporting line: an approval by a reviewer who has a manager above them in the reviewer pool moves the idea to that manager rather than closing it, and each hop is written to the idea's workflow trail. Points are awarded on submission, on approval and on implementation, at values that are configuration rather than code.

2.4 The life of one request

Almost every case in this document is an assertion about one of the steps below, so it is worth setting them out once. A request arrives and passes through, in order:

- **Security headers** - a locked-down content policy, no-referrer, nosniff, no framework fingerprint, and HSTS when HTTPS is enforced.
- **Transport** - in a configuration that enforces HTTPS, a plaintext request is redirected rather than allowed to carry a bearer token in the clear.
- **Cross-origin policy** - only origins on the configured allow-list are granted access.
- **Body limits** - one megabyte, enforced before any handler sees the request, with a clean client error rather than a server error.

- **Rate limiting** - a per-IP global budget, a much tighter budget on authentication endpoints that counts only failures, and a small hourly budget on expensive operations such as bulk rescoring and imports.
- **Authentication** - the signed token is verified with the algorithm pinned by the server, then the tenant is resolved from the organisation slug inside it and that tenant's connection pool is attached to the request.
- **Live user check** - the user row is re-read: role, account status and the password-change marker come from the database, never from the token. A user still holding a temporary password is confined to changing it.
- **Authorisation** - the route's declared role list is checked against the role just read from the database.
- **Handler and service** - all statements are parameterised, and every query runs against the tenant pool that was attached in step six, which is what makes cross-tenant access structurally impossible rather than merely forbidden.
- **Error handling** - known errors keep their status and message; anything unexpected is logged with its method and path and returned to the caller as a generic message with no internals.

3. Test environment and method

The suite is not a set of unit tests with mocked databases. It starts the real application factory, listens on a real port, and sends real HTTP requests. Where a case needs to know what was stored, it queries the database directly rather than trusting the response body it just received.

3.1 Environment

Application	Node.js / Express API (the production application factory, unmodified)
Database	MySQL, three scratch schemas created and dropped by the run: a tenant registry and two tenant databases, plus any tenant the product itself provisions during the run
Isolation	Registry, fallback tenant and all seed rows are redirected to the scratch schemas before the application configuration is loaded, so no code path - including the default-tenant fallback - can reach a real database
Second instance	For the horizontal-scalability module a second, separate operating-system process runs the same application against the same databases, on its own port
External systems	The QCMS endpoint is replaced by purpose-built local servers that reproduce its documented behaviour, including 201 imported, 409 duplicate, 401 invalid key, a connection refusal and a server that accepts the connection and then never answers
Mail	Deliberately pointed at an unreachable SMTP host for the fault-tolerance cases
Data volume	One tenant is loaded with 5,000 ideas and 300 additional user accounts before the performance cases run
Rate limits	The per-IP global limiter is raised for the run because the suite is itself a load generator; the shipped defaults are 300 requests/minute per IP and 30 authentication attempts per 15 minutes

3.2 What "Actual Output" is allowed to be

A case may only report what it observed. Latency figures are wall-clock measurements around the request. Row counts come from a direct query. Error strings are the exact text the API returned. Where a case documents a design limitation rather than a defect - for example that per-process rate-limit counters are not shared between instances - the limitation is written into the expected output so the reader is not misled by a green result.

3.3 Reproducing this run

```
cd backend npm install node test/tc_runner.mjs # writes test/tc_results.json npm test # the
assertion suite (33 tests) python ../docs/gen_testcases_doc.py
```

The runner is destructive only towards its own scratch schemas, which it drops on the way in and on the way out. It is safe to run on a machine that also holds live data.

3.4 The rules a case has to follow

A suite is only as trustworthy as the discipline behind the cases in it, so five rules are applied to every one of them.

- **The expectation is written first.** If the expected output were written after seeing the behaviour, the suite would document the system rather than test it.
- **Assert the consequence, not the mechanism.** A vote case counts the stored rows; it does not check that a particular function was called. That is what lets the implementation be rewritten without rewriting the suite.
- **Verify persistence independently.** Where a case matters, it queries the database rather than believing the response body it just received - several cases pass their HTTP check and would still fail here.
- **No case may depend on another case's leftovers.** Cases that must deactivate, delete, lock or expire an account seed their own throwaway account for the purpose.
- **A limitation is written into the expectation.** Where behaviour is a deliberate design trade-off rather than a guarantee - per-process rate-limit counters, attachments on local storage - the expected output says so, so a green result never overstates what was proven.

3.5 Threat model behind the safety cases

The safety module is not a checklist of fashionable vulnerabilities; it is derived from what an attacker or a careless integration would actually try against a multi-tenant workplace application, and from the incidents that occur in systems of this shape.

Threat	Why it matters here	Cases
Forged or edited session token	Anyone who can mint a token owns every tenant. This includes the classic "algorithm none" trick and simply editing the role inside an otherwise valid token.	TC-SAFE-001 to TC-SAFE-004
Stale session after a personnel change	An employee who leaves, or is demoted, keeps a valid token for hours unless the server re-checks the row.	TC-SAFE-007 to TC-SAFE-010
Self-granted privilege	The cheapest escalation is to send a role field the server did not expect to receive.	TC-SAFE-005, TC-SAFE-006
Cross-tenant read	Identifiers auto-increment per tenant, so the same number exists in every organisation. Isolation must not depend on a filter somebody could forget.	TC-SEC-001, TC-SUP-002, TC-EXP-003, TC-SAFE-023
Attachment as an attack surface	Uploads are the most common way to get executable content into an origin, and the most common way for private documents to become public URLs.	TC-SAFE-017 to TC-SAFE-026
Injection	Any string a user types can reach a query or a rendered page.	TC-AUTH-007, TC-SEC-003, TC-SAFE-015, TC-SAFE-016
Secret disclosure	Password hashes, mail passwords and integration keys leak through responses far more often than through databases.	TC-SAFE-011, TC-BRND-003, TC-QCMS-002
Identity disclosure of an anonymous report	The feature is worthless if a colleague can work out who filed the report - and the disclosure is usually in a field nobody thought about.	TC-SAFE-027 to TC-SAFE-029
Credential stuffing and spraying	Per-account lockout stops grinding one account; the IP budget is what caps a spray across many.	TC-AUTH-011, TC-SAFE-033, TC-SAFE-034, TC-SCLH-011
Denial of service through cost	Oversized bodies, huge uploads, unbounded exports and repeated expensive operations.	TC-SEC-004, TC-SAFE-019, TC-SCLV-002, TC-SCLV-003

Threat	Why it matters here	Cases
Insecure deployment	The most likely real-world failure is not an exploit but shipping with the example secret and a passwordless database.	TC-SAFE-035, TC-OPS-007

4. Test approach, dimension by dimension

4.1 Functional behaviour

Every feature area is exercised through the same endpoints the browser uses: authentication, organisation provisioning, user administration and bulk import, idea submission and retrieval, voting and rating, comments, the review and approval chain, categories and challenges, analytics, notifications, exports, branding and mail settings, the support desk, and the QCMS integration. Each area includes at least one case that supplies invalid input and at least one that supplies valid input from the wrong role.

4.2 Safety, security and privacy

This is the largest module in the suite. It is organised around the ways a system of this kind is actually attacked or, more often, accidentally leaks:

- **Token forgery and misuse** - unsigned (alg=none) tokens, tokens signed with a foreign secret, expired tokens, and tokens whose embedded role has been edited upwards.
- **Session lifetime against reality** - a token must stop working the moment the account is deactivated, deleted, or has its password changed. Offboarding that does not end a session is a real breach, not a nuisance.
- **Privilege escalation and mass assignment** - an employee editing their own user record, or posting a role and a points balance through the profile endpoint.
- **Tenant isolation** - one organisation presenting another organisation's identifiers for ideas, tickets, attachments and PDFs.
- **File handling** - extension allow-list, double extensions, oversize uploads, ownership on upload and delete, authentication on download, path traversal, and the response headers that stop a stored file executing in the application's own origin.
- **Injection and malformed input** - SQL metacharacters in query parameters and stored fields, script payloads, malformed JSON, oversized bodies, CRLF in user-supplied names.
- **Secret handling** - password hashes, SMTP passwords and integration API keys must never appear in any response, and the configuration guard must refuse to boot a production server with placeholder secrets.
- **Privacy of anonymous submissions** - the identity must be hidden from colleagues in every field of the response, while remaining recoverable in the database for accountability.
- **Brute force** - per-account logout that a correct password cannot clear, counted centrally so that it cannot be evaded by rotating between application instances.

4.3 Reliability and fault tolerance

Two questions: does the service degrade gracefully when something it depends on fails, and does it stay correct when several things happen at the same instant. The first is tested by breaking dependencies deliberately - an unreachable QCMS endpoint, a QCMS server that accepts a connection and never replies, an unreachable mail server - and requiring that the user-facing operation still completes and is still recorded. The second is tested with genuine parallel requests: ten simultaneous votes from one user, ten simultaneous comments, five simultaneous approvals of the same idea, and two simultaneous account creations with the same email address. Concurrency cases are where this run found real defects; section 6 records them.

4.4 Scalability - horizontal

Horizontal scalability means adding a second application server and having the system behave identically. The only honest way to test that is to run two separate processes against one database and prove that a request may land on either. The suite spawns a second instance and checks that a session minted on one is accepted by the other in both directions, that data written on one is visible on the other with no cache to go stale, that organisation settings, branding, integration credentials and newly created tenants are picked up without a restart, that the brute-force lockout is enforced across instances, and that killing one instance leaves the other serving.

4.5 Scalability - vertical

Vertical scalability means the same box coping with far more data. One tenant is loaded with five thousand ideas and three hundred extra users, and then every screen that an organisation actually uses is held to a latency budget on that dataset: the idea list, the dashboard aggregates, analytics, the leaderboard, the audit report, search, filtered lists, the integration listing, single-idea retrieval and the full CSV export. Alongside the budgets, the suite checks the structural properties that keep those numbers flat as the dataset grows: the response is capped at one hundred rows and a client cannot raise that cap, the ordering index exists and the query plan uses it, thirty concurrent requests queue against a pool of ten rather than failing, resident memory does not grow with the dataset, and the second tenant is unaffected by the first tenant's volume.

4.6 Data integrity and recovery

Schema changes are applied by a migration runner with a ledger, so the suite runs it, runs it again to prove it applies nothing the second time, and then drops a brand-new migration file into the folder to prove that a future change is picked up automatically and recorded. Beyond that: unique constraints, cascade behaviour when a parent row is removed, the atomicity of a decision (status and audit entry together), points arithmetic against the configured values, timestamp consistency with the database clock, character set and storage engine of every table, a primary key on every table, and the backup and restore path.

4.7 Extensibility and future scope

A test suite cannot prove that a future requirement will be easy, but it can prove that the extension points exist and work today. The cases here add a real tenant at runtime and use it immediately, hold two organisations on different integration endpoints at the same time, confirm that every capacity and policy limit is read from configuration rather than hard-coded, check translation coverage across all seven shipped languages, apply a new migration file, and confirm that the field mapping to the downstream system is a pure function - which is what makes a second downstream system a new mapper rather than a new project.

4.8 Observability and operations

Liveness and readiness are separate probes, because a process that is running but cannot reach its database is worse than one that is down. The suite checks both, the cost of polling them, the graceful-shutdown path, the crash handlers, the durable log trail, the audit report, and the boot-time configuration guard.

4.9 What this suite does not cover

Stating the boundary is part of the evidence. This suite does not exercise the browser: layout, keyboard navigation, screen-reader behaviour and visual regressions are outside its reach, and the rules it verifies are verified on the server precisely because a browser cannot be trusted to enforce them. It does not test a real SMTP delivery, a real QCMS installation, or the network between them - it tests what this system does when those behave, misbehave, or vanish. It is not a penetration test by an independent party, and it is not a capacity certification on production hardware; section 9 says what to do about both.

5. Results

The run named on the cover executed **228** cases. **228** passed and **0** failed, a pass rate of **100.0%**. The failures found during development of this cycle are described in section 6 together with the code changes that resolved them; the figures below are from the final run after those changes.

5.1 By assurance dimension

Assurance dimension	Cases	Passed	Failed	Pass rate
Functional behaviour	79	79	0	100%
Safety, security and privacy	45	45	0	100%
Reliability and fault tolerance	24	24	0	100%
Scalability - horizontal	16	16	0	100%
Scalability - vertical	21	21	0	100%
Data integrity and recovery	17	17	0	100%
Extensibility and future scope	18	18	0	100%
Observability and operations	8	8	0	100%
All dimensions	228	228	0	100.0%

5.2 By module

Module	Dimension	Cases	Passed	Failed
Authentication	Functional behaviour	12	12	0
Platform Admin	Functional behaviour	10	10	0
Org Admin / Users	Functional behaviour	6	6	0
Ideas	Functional behaviour	8	8	0
Voting & Community	Functional behaviour	6	6	0
Comments	Functional behaviour	5	5	0
Review & Approval	Functional behaviour	5	5	0
Categories & Challenges	Functional behaviour	4	4	0
Analytics & Reports	Functional behaviour	4	4	0
Notifications	Functional behaviour	3	3	0
Reports & Export	Functional behaviour	3	3	0
Branding & Settings	Functional behaviour	3	3	0
Support	Functional behaviour	3	3	0
QCMS Integration	Functional behaviour	7	7	0
Security & Multi-Tenancy	Safety, security and privacy	5	5	0
Safety & Data Protection	Safety, security and privacy	40	40	0
Reliability & Fault Tolerance	Reliability and fault tolerance	24	24	0
Scalability - Horizontal	Scalability - horizontal	16	16	0
Scalability - Vertical & Performance	Scalability - vertical	21	21	0

Module	Dimension	Cases	Passed	Failed
Data Integrity & Recovery	Data integrity and recovery	17	17	0
Extensibility & Future Scope	Extensibility and future scope	18	18	0
Observability & Operations	Observability and operations	8	8	0

6. Defects found by this cycle, and their fixes

Deepening the suite was worthwhile precisely because it failed. Three genuine defects were found, all of them in behaviour that the previous, shallower suite could not have reached: two only appear under true concurrency, and one only appears when the response is inspected field by field rather than as a whole. Each was fixed in the product, and each now has a permanent case guarding it.

6.1 Simultaneous submissions collided on the idea code

Severity	High - user-visible failure of the core action
Found by	TC-REL-024 (export requested while writes are in flight): only 2 of 5 parallel submissions were stored.
Symptom	Two employees submitting at the same instant both received a server error. The idea was not saved and the submitter was given no usable explanation.
Root cause	The next idea code was derived from COUNT(*) of this year's ideas. Two concurrent submissions read the same count and computed the same code, and the column is UNIQUE, so the second INSERT failed. The same arithmetic also reused a number whenever any idea was deleted, so a deletion could break the next unrelated submission.
Fix	The sequence is now taken from the highest code already issued this year rather than a count, and the insert retries on a duplicate-key collision after re-reading the sequence. A collision is now invisible to the submitter instead of being fatal.
Now guarded by	TC-REL-024 (five parallel writes must all be stored) and TC-DATA-008 (the sequence must continue past the highest issued code after a deletion).

6.2 An idea could be approved five times over

Severity	Medium - audit and notification integrity
Found by	TC-REL-010 (five simultaneous approvals of the same idea): five approval entries were written.
Symptom	A double-tapped Approve button, or a client retry on a slow connection, produced multiple approval records for one decision, multiple notifications to the submitter and a duplicated audit trail.
Root cause	The duplicate-action guard read the recent-actions table and then wrote to it. Under true concurrency all callers read "nothing recent" before any of them wrote, so the guard never fired.
Fix	The whole decision now runs under a named database lock scoped to the idea, so the read-then-write is atomic - and, because the lock lives in MySQL rather than in the process, it holds across every application instance behind a load balancer, not just within one.
Now guarded by	TC-REL-010 (exactly one approval row after five parallel approvals) and TC-DATA-009 (status and audit entry written together).

6.3 Anonymous submissions were identifiable from the approval timeline

Severity	High - privacy commitment broken
Found by	TC-SAFE-027 (anonymous idea opened by a colleague): the author's real name was present in the response.
Symptom	The header fields of an anonymous idea were masked, but the approval timeline attached to the same response carried an actor name on every entry - starting with the author's own "Submitted" entry - and the co-suggester list named the colleagues who raised it with them. Any employee who opened the idea could see exactly who filed the anonymous report.
Root cause	Masking was applied to the summary fields only, and had not followed the data into the related collections returned alongside it.

Fix	Masking now covers the whole payload for a viewer who is not privileged and is not the author: timeline entries authored by the submitter are attributed to "Anonymous", and the co-suggester list is withheld. The author still sees their own submission unmasked, and the submitter identity remains in the database for accountability.
Now guarded by	TC-SAFE-027 (colleague must not see the identity anywhere in the payload), TC-SAFE-028 (the author must still see their own), TC-SAFE-029 (the database must still hold the real submitter).

6.4 Observations that were not defects

- The global rate limiter did its job so effectively that it initially failed the performance module - the suite itself was throttled at 300 requests per minute. The limiter is correct; the runner now raises the limit for its own traffic and the shipped default is documented rather than disabled.
- Tenant provisioning is not idempotent when a database for that slug already exists from an earlier run. This is correct behaviour for a product (silently adopting an existing schema would be worse), but it means an aborted provisioning attempt must be cleaned up before the slug can be reused. The runner now removes the schemas it provisions.

7. Measured behaviour under load

The figures below were measured during this run on a single developer-class machine with the database on the same host. They are not a capacity certification for production hardware; they are evidence that the work per request does not grow with the size of the dataset, which is the property that decides whether a system survives its third year.

7.1 Dataset

Load applied	5023 ideas in 1183 ms (4227 rows/s)
Idea list payload	100 rows returned, payload 200 KB
Client cap override attempt	100 rows returned
Query plan for the list	type=index, key=idx_ideas_updated_at, rows=100, extra=-
Ordering index	Index present on column updated_at

7.2 Latency at 5,000 ideas

Operation	Measured	Budget	Result
Idea list latency under load	79 ms	3000 ms	Pass
Dashboard aggregate latency under load	21 ms	3000 ms	Pass
Analytics latency under load	33 ms	4000 ms	Pass
Leaderboard latency with 300+ users	60 ms	3000 ms	Pass
Audit report latency under load	7 ms	4000 ms	Pass
Title search latency under load	86 ms	3000 ms	Pass
Status-filtered list latency under load	26 ms	3000 ms	Pass
Approved-ideas (integration) listing under load	36 ms	4000 ms	Pass
Single idea detail stays constant-time	26 ms	1000 ms	Pass
Employee's own idea list under load	254 ms	3000 ms	Pass

7.3 Throughput, concurrency and footprint

Full CSV export	5024 lines, 0.6 MB in 55 ms
Concurrency against the pool	30/30 succeeded in 201 ms with DB_POOL_SIZE=10
Sustained read throughput	100 requests in 1805 ms -> 55.4 req/s, 0 errors
Neighbouring tenant	Org B dashboard 200 in 13 ms
Process memory after the load	RSS 180 MB, heap 87 MB after 5,000 ideas

The list endpoint returns at most one hundred rows and the client cannot raise that cap, so the payload is constant whether the organisation holds five thousand ideas or five hundred thousand. The ordering index turns the top-100 read into an ordered index scan that stops at the limit - visible in the recorded query plan above - rather than a full scan and a sort of the whole table. Resident memory after loading and querying the dataset shows that no part of it is held in the application.

7.4 Two instances behind one database

Property	Observed
A second application instance starts against the same databases	Instance 2 on http://127.0.0.1:55460, health=200
Session issued by instance 1 used on instance 2	Status 200
Session issued by instance 2 used on instance 1	login on 2 = ok, request on 1 = 200
Idea created on instance 1 read from instance 2	Status 200
Organisation settings changed on 1 take effect on 2	Instance 2 reports review_sla_days=11
Failed logins counted on 1 lock the account on 2	Instance 2 also blocked: Too many failed attempts. Please try again in 15 minute(s).
Organisation created on 1 is served by 2 with no restart	create=200, login on instance 2=ok
Twenty requests alternating between the two instances	20/20 succeeded across both instances
One instance is killed mid-service	instance 1 = 200 after instance 2 was killed (instance 2 = down)

Nothing that matters is held in a process. Sessions are signed tokens re-validated against the database on every request; the brute-force lockout, the tenant registry, organisation settings, branding and integration credentials are all shared state in MySQL. Two limitations are recorded honestly rather than hidden: uploaded files are written to a local directory, so a multi-host deployment needs shared or object storage; and the per-IP rate-limit counters are per process, so a strict global budget needs a shared store. Both are noted in section 9.

7.5 What the measurements imply for sizing

Three properties observed above are what make a sizing estimate possible at all. First, the work per read is bounded: the list is capped at one hundred rows and served from an ordered index, so its cost is independent of how many ideas the organisation has accumulated. Second, the application holds no dataset in memory, so a larger organisation costs database storage rather than application memory. Third, tenants are separate schemas, so one large customer does not slow another - demonstrated by the neighbouring organisation staying fast while the first held five thousand ideas.

Together those mean capacity is planned in three independent directions rather than one: application instances for concurrent users, database resources for total data, and - beyond a certain size - additional database hosts for particularly large tenants, which the registry already supports by storing a host per tenant. The suggested starting points below follow from the measurements; they are a planning aid, not a guarantee, and section 9 recommends re-running the vertical module on the target hardware as an acceptance step.

Application instances	Two behind a load balancer as a minimum, for rolling restarts and fault tolerance - proven interchangeable in section 7.4. Add instances for concurrent users, not for data volume.
Database connections	Per-tenant pool size multiplied by (tenants + 1) must stay below the server's connection limit. Thirty concurrent requests queued cleanly against a pool of ten in this run.
Storage	Ideas are small rows; attachments dominate. Budget from expected attachments per idea and the configured maximum file size, on shared or object storage once there is more than one host.
Backups	A per-schema dump plus the uploads directory, retained on the schedule in the deployment guide.
Growth trigger	Move a tenant to its own database host when its schema alone drives the database server, rather than splitting the application.

8. Coverage matrix

Read down the column to see how much of the suite defends each property; read across to see which modules contribute. A blank cell is not necessarily a gap - tenant isolation, for example, is concentrated in the safety module by design - but it is where the next cycle should look.

Module	Valid input	Invalid input	Wrong role	Tenant boundary	Concurrency	Failure injection	Latency budget	Persistence check
Authentication	X	X	X	X				X
Platform Admin	X	X	X	X				X
Org Admin / Users	X	X	X		X			X
Ideas	X	X	X	X	X		X	X
Voting & Community	X	X	X		X			X
Comments	X	X			X			X
Review & Approval	X	X	X		X			X
Categories & Challenges	X		X					
Analytics & Reports	X		X				X	
Notifications	X		X					
Reports & Export	X		X	X	X		X	
Branding & Settings	X		X					X
Support	X		X	X				
QCMS Integration	X	X	X	X		X	X	X
Security & Multi-Tenancy		X	X	X				
Safety & Data Protection	X	X	X	X		X		X
Reliability & Fault Tolerance	X	X			X	X		X
Scalability - Horizontal	X			X	X	X		X
Scalability - Vertical & Performance	X	X		X	X		X	X
Data Integrity & Recovery	X	X		X		X		X
Extensibility & Future Scope	X			X				X
Observability & Operations	X		X				X	

9. Residual risks, limitations and recommendations

Everything in this section passed its test. They are listed because a green result under one set of assumptions is a risk under another, and the reader deserves to know which assumptions were made.

Area	What is true today	Recommendation before scale-out
Uploaded files	Attachments are written to a local directory per tenant and served only through an authenticated, tenant-scoped endpoint. Both instances in the horizontal test served the same file because they shared a filesystem.	On more than one host, mount shared storage or move attachments to object storage. Nothing else in the design needs to change.
Rate limiting	Counters live in each process. Two instances therefore allow twice the configured budget.	Point the limiter at a shared store (for example Redis) if a strict global budget is a requirement. The per-account brute-force lockout is already central and is unaffected.
Read pagination	List endpoints are capped at 100 rows, which keeps payloads and query cost constant.	Add cursor pagination when a screen genuinely needs to walk beyond the cap; the cap is server-side so no client is broken by adding it.
Uploaded file content	Uploads are validated by extension allow-list and size, stored under a generated name, and always served as an attachment with nosniff, so a stored file cannot execute in the application origin.	Add content-signature checking and an anti-virus pass if the deployment accepts documents from outside the organisation.
Database capacity	One schema per tenant, with the host recorded per tenant in the registry.	Growth is absorbed by moving tenants to additional database hosts - already supported by the registry - before any single schema becomes a bottleneck.
Performance figures	Measured on a developer machine with the database on the same host.	Re-run the vertical module on the target hardware as an acceptance step; the budgets in section 7 are the ones to hold to.
Secrets in production	The application refuses to boot in production with a placeholder token secret or a passwordless database account.	Keep that guard enabled and rotate the token secret on a schedule; rotation invalidates existing sessions by design.
Front-end coverage	The suite exercises the API that every screen uses, and the server enforces every rule independently of the browser.	Add a browser-level regression pass for visual and interaction defects, which no API-level suite can see.

9.1 Future scope, and what already supports it

The extensibility module is written so that each case names the extension it protects. In summary:

- A new customer organisation is created at runtime and is usable immediately - no deployment, no restart.
- An eighth language is one dictionary file and one registry line; all seven shipped locales are complete today.
- A new feature area is one route module registered in one aggregator.
- A new role is an entry in a declared list rather than a scattered set of conditionals.
- A second downstream system is a new mapping function: the QCMS mapping is pure, with no I/O of its own.
- Capacity and policy limits - rate limits, pool size, token lifetime, file size, password minimum and every points value - are environment configuration.
- Schema evolution is a numbered file in the migrations folder, applied and ledgered automatically, proven in this run by adding one.

10. Test cases in full

Every case executed in the run named on the cover, in execution order, grouped by module. Actual Output is verbatim.

10.1 Authentication (12/12 passed)

The front door. These cases cover the three ways in (organisation admin, employee, vendor platform admin), every way of getting it wrong, and the properties that must hold while getting it wrong: an unknown email and a wrong password must be indistinguishable to the caller, injection strings must be inert, a deactivated account must not open, a tampered or absent token must be refused, repeated failures must throttle the account, and a forgotten-password request must not reveal whether the address exists.

Test Case ID	Module	Functionality	Expected Output	Actual Output	Result	Timestamp
TC-AUTH-001	Authentication	Org Admin logs in with correct email/password	Login succeeds, JWT issued, role = admin	Login ok, role=admin	Pass	2026-07-28 17:34:45
TC-AUTH-002	Authentication	Employee logs in with correct credentials	Login succeeds, role = employee	Login ok, role=employee	Pass	2026-07-28 17:34:45
TC-AUTH-003	Authentication	Platform Admin logs in with correct credentials	Login succeeds, role = platform_admin	role=platform_admin	Pass	2026-07-28 17:34:45
TC-AUTH-004	Authentication	User enters correct email but wrong password	Login blocked, generic error, no token	Blocked 401: Invalid email/phone or password. 4 attempt(s) remaining.	Pass	2026-07-28 17:34:45
TC-AUTH-005	Authentication	User enters an email that does not exist	Same generic error as wrong-password (no account-existence leak)	Identical generic message (401)	Pass	2026-07-28 17:34:46
TC-AUTH-006	Authentication	User submits login with empty email/password	Validation error, login blocked (not a 500)	Rejected 400: Email and password are required.	Pass	2026-07-28 17:34:46
TC-AUTH-007	Authentication	SQL injection string in the email field	Safely rejected, no error/crash exposed	Safe 401	Pass	2026-07-28 17:34:46
TC-AUTH-008	Authentication	Deactivated user tries to log in with correct credentials	Login blocked, account-inactive handled	Blocked 401: Invalid email/phone or password. 4 attempt(s) remaining.	Pass	2026-07-28 17:34:47
TC-AUTH-009	Authentication	Tampered JWT used on a protected endpoint	Rejected 401 - invalid signature grants no access	Status 401	Pass	2026-07-28 17:34:47
TC-AUTH-010	Authentication	Access a protected endpoint with no token at all	Rejected 401	Status 401	Pass	2026-07-28 17:34:47
TC-AUTH-011	Authentication	Repeated wrong-password attempts on one account	Attempts are counted and the account is throttled/locked	Throttle msg: Too many failed attempts. Please try again in 15 minutes.	Pass	2026-07-28 17:34:47
TC-AUTH-012	Authentication	Forgot-password request for an unknown email	Generic success, no account-existence leak	Status 200: ok	Pass	2026-07-28 17:34:47

10.2 Platform Admin (10/10 passed)

Vendor-side administration. Creating an organisation provisions a database, a schema and a first administrator in one action, so the cases cover the happy path, duplicate slugs, missing fields, suspension and reactivation, the confirmation guard on deletion, password-reset targeting, and the boundary that stops a tenant administrator reaching any of it.

Test Case ID	Module	Functionality	Expected Output	Actual Output	Result	Timestamp
TC-PLAT-001	Platform Admin	Create a new organization with valid details	Tenant + org-admin account created	Status 200: created	Pass	2026-07-28 17:34:49
TC-PLAT-002	Platform Admin	Create an organization with a duplicate slug	Rejected - slug already in use	Rejected 409: Organization code already in use.	Pass	2026-07-28 17:34:49
TC-PLAT-003	Platform Admin	Create an organization with missing required fields	Validation error listing missing fields	Rejected 400: All fields are required.	Pass	2026-07-28 17:34:49
TC-PLAT-004	Platform Admin	List all tenants	Returns the tenant registry	Status 200, 3 tenants	Pass	2026-07-28 17:34:49
TC-PLAT-005	Platform Admin	Suspend a tenant then reactivate it	Status transitions suspended -> active	suspend=200, reactivate=200	Pass	2026-07-28 17:34:49
TC-PLAT-006	Platform Admin	Delete a tenant with a wrong confirmation slug	Blocked - confirmation mismatch	Blocked 400: Type the org code "orgb" to confirm deletion.	Pass	2026-07-28 17:34:49
TC-PLAT-007	Platform Admin	Reset a tenant admin password targeting a non-admin email	Rejected - target is not that org's admin	Rejected 404: No admin account with that email in this organisation.	Pass	2026-07-28 17:34:49
TC-PLAT-008	Platform Admin	Org Admin tries to reach the platform tenants endpoint	Forbidden - platform scope only	Status 401	Pass	2026-07-28 17:34:49
TC-PLAT-009	Platform Admin	Create a platform admin with a weak password	Rejected for not meeting policy	Rejected 400: Password must be at least 12 characters.	Pass	2026-07-28 17:34:49
TC-PLAT-010	Platform Admin	Platform admin tries to delete their own account	Blocked - cannot remove self	Blocked 400: You cannot delete your own account.	Pass	2026-07-28 17:34:49

10.3 Org Admin / Users (6/6 passed)

Account lifecycle inside one organisation: creating users with derived temporary passwords, rejecting duplicates and missing fields, listing the organisation, previewing a bulk spreadsheet import with a deliberately malformed row, and refusing all of it to a non-administrator.

Test Case ID	Module	Functionality	Expected Output	Actual Output	Result	Timestamp
TC-USER-001	Org Admin / Users	Admin creates a user with valid details	User created with a derived temporary password	Status 200: created	Pass	2026-07-28 17:34:49

Test Case ID	Module	Functionality	Expected Output	Actual Output	Result	Timestamp
TC-USER-002	Org Admin / Users	Create a user with a missing email	Validation error - email required	Rejected 400: Name, email, and employee ID are required.	Pass	2026-07-28 17:34:49
TC-USER-003	Org Admin / Users	Create a user with an email already in use	Rejected - email already registered	Rejected 409: Email or employee ID already exists.	Pass	2026-07-28 17:34:49
TC-USER-004	Org Admin / Users	Employee (non-admin) tries to create a user	Forbidden - admin only	Status 403	Pass	2026-07-28 17:34:49
TC-USER-005	Org Admin / Users	Admin lists users in the organization	Returns the org user list	Status 200, 3 users	Pass	2026-07-28 17:34:49
TC-USER-006	Org Admin / Users	Bulk-import preview with a malformed row	Row-level error reported, not a crash	Status 200: previewed	Pass	2026-07-28 17:34:49

10.4 Ideas (8/8 passed)

The core action of the product. Submission with valid content, rejection of missing and absurd content, retrieval by identifier, the organisation list, the dashboard aggregates, and the requirement that none of it is reachable without a session.

Test Case ID	Module	Functionality	Expected Output	Actual Output	Result	Timestamp
TC-IDEA-001	Ideas	Employee submits a valid idea	Idea accepted, id returned	Status 200: id=3	Pass	2026-07-28 17:34:49
TC-IDEA-002	Ideas	Submit an idea with a missing title	Validation error - title required	Rejected 400: Title, present situation and proposed solution are required.	Pass	2026-07-28 17:34:49
TC-IDEA-003	Ideas	Submit an idea with a <script> tag in the title	Stored safely / escaped, no script execution, no crash	Status 200 (stored)	Pass	2026-07-28 17:34:49
TC-IDEA-004	Ideas	Submit an idea with an extremely long title (5000+ chars)	Rejected with validation, not a server crash	Handled 200: accepted	Pass	2026-07-28 17:34:49
TC-IDEA-005	Ideas	Fetch own idea by id	Returns the idea record	Status 200	Pass	2026-07-28 17:34:49
TC-IDEA-006	Ideas	List ideas	Returns a list	Status 200	Pass	2026-07-28 17:34:49
TC-IDEA-007	Ideas	Dashboard aggregates for the org	Returns status counts / totals	Status 200	Pass	2026-07-28 17:34:49
TC-IDEA-008	Ideas	Submit an idea without authentication	Rejected 401	Status 401	Pass	2026-07-28 17:34:49

10.5 Voting & Community (6/6 passed)

Community signals on an idea: upvotes that toggle rather than accumulate, ratings bounded to one to five, rejection of invalid vote types, and authentication on every path.

Test Case ID	Module	Functionality	Expected Output	Actual Output	Result	Timestamp
TC-VOTE-001	Voting & Community	Community upvote on an idea	Upvote recorded	Status 200	Pass	2026-07-28 17:34:49
TC-VOTE-002	Voting & Community	Same user upvotes the same idea again	Vote toggles off (idempotent), no double count	Status 200	Pass	2026-07-28 17:34:49
TC-VOTE-003	Voting & Community	Rate an idea 1-5	Rating stored	Status 200: rated	Pass	2026-07-28 17:34:49
TC-VOTE-004	Voting & Community	Rate an idea with an out-of-range value (6)	Rejected - rating must be 1-5	Rejected 400: Invalid request - idea_id and rating (1-5) required.	Pass	2026-07-28 17:34:49
TC-VOTE-005	Voting & Community	Community vote with an invalid vote_type	Rejected - vote_type must be up/down	Rejected 400: idea_id and vote_type (up/down) required.	Pass	2026-07-28 17:34:49
TC-VOTE-006	Voting & Community	Vote without a token	Rejected 401	Status 401	Pass	2026-07-28 17:34:49

10.6 Comments (5/5 passed)

Discussion on an idea, with the boundaries that keep it usable: no empty comments, a length cap, a real idea to attach to, and safe handling of markup.

Test Case ID	Module	Functionality	Expected Output	Actual Output	Result	Timestamp
TC-CMNT-001	Comments	Add a comment to an idea	Comment created	Status 200: added	Pass	2026-07-28 17:34:49
TC-CMNT-002	Comments	Add an empty comment	Rejected - content required	Rejected 400: Comment content cannot be empty.	Pass	2026-07-28 17:34:49
TC-CMNT-003	Comments	Add a comment over 1000 characters	Rejected - length cap enforced	Rejected 400: Comment cannot exceed 1000 characters.	Pass	2026-07-28 17:34:49
TC-CMNT-004	Comments	Comment on a non-existent idea	Not found	Status 404	Pass	2026-07-28 17:34:49
TC-CMNT-005	Comments	Comment containing a <script> tag	Stored safely / escaped on render, no crash	Status 200	Pass	2026-07-28 17:34:49

10.7 Review & Approval (5/5 passed)

The decision path. A reviewer approves; the submitter cannot approve their own idea; an invalid decision value is refused; a decision on a non-existent idea is a clean not-found; and a repeated identical decision does not duplicate the audit trail.

Test Case ID	Module	Functionality	Expected Output	Actual Output	Result	Timestamp
TC-RVW-001	Review & Approval	Reviewer approves an idea	Idea moves to Approved with a workflow entry	Status 200: approved	Pass	2026-07-28 17:34:49
TC-RVW-002	Review & Approval	Submitter tries to approve their own idea	Forbidden - cannot review own idea	Status 403: Insufficient permissions	Pass	2026-07-28 17:34:49
TC-RVW-003	Review & Approval	Review with an invalid decision value	Rejected - invalid decision	Rejected 400: Invalid request.	Pass	2026-07-28 17:34:49
TC-RVW-004	Review & Approval	Approve a non-existent idea	Not found	Status 404	Pass	2026-07-28 17:34:49
TC-RVW-005	Review & Approval	Duplicate identical approval within 10 seconds	Idempotency guard - no duplicate workflow entry	Approved workflow rows: 1	Pass	2026-07-28 17:34:49

10.8 Categories & Challenges (4/4 passed)

The taxonomy an organisation shapes for itself - categories and time-boxed campaigns - and the administrative boundary around changing it.

Test Case ID	Module	Functionality	Expected Output	Actual Output	Result	Timestamp
TC-CAT-001	Categories & Challenges	Admin creates a category	Category created	Status 200: created	Pass	2026-07-28 17:34:49
TC-CAT-002	Categories & Challenges	List categories	Returns the category list	Status 200	Pass	2026-07-28 17:34:49
TC-CAT-003	Categories & Challenges	Non-admin tries to create a category	Forbidden - admin only	Status 403	Pass	2026-07-28 17:34:49
TC-CAT-004	Categories & Challenges	Admin creates a challenge/campaign	Challenge created	Status 200: created	Pass	2026-07-28 17:34:49

10.9 Analytics & Reports (4/4 passed)

Aggregates for management: analytics, the audit report, the analytics export and the leaderboard.

Test Case ID	Module	Functionality	Expected Output	Actual Output	Result	Timestamp
TC-ANL-001	Analytics & Reports	Admin loads analytics aggregates	Returns aggregate metrics	Status 200	Pass	2026-07-28 17:34:49
TC-ANL-002	Analytics & Reports	Admin loads the audit report	Returns the audit trail	Status 200	Pass	2026-07-28 17:34:49
TC-ANL-003	Analytics & Reports	Analytics CSV export	Returns a downloadable analytics export	Status 200, type=text/html; charset=utf-8	Pass	2026-07-28 17:34:49

Test Case ID	Module	Functionality	Expected Output	Actual Output	Result	Timestamp
TC-ANL-004	Analytics & Reports	Leaderboard for the org	Returns a ranked list	Status 200	Pass	2026-07-28 17:34:49

10.10 Notifications (3/3 passed)

The in-product signal that something needs attention, including the requirement that marking read is idempotent and that the list is private to its owner.

Test Case ID	Module	Functionality	Expected Output	Actual Output	Result	Timestamp
TC-NTF-001	Notifications	List notifications for the signed-in user	Returns the notification list	Status 200	Pass	2026-07-28 17:34:49
TC-NTF-002	Notifications	Mark notifications read	Marked read, ok	Status 200	Pass	2026-07-28 17:34:49
TC-NTF-003	Notifications	List notifications without a token	Rejected 401	Status 401	Pass	2026-07-28 17:34:49

10.11 Reports & Export (3/3 passed)

Data leaving the system: the full CSV export, the single-idea closure PDF, and the tenant boundary on that PDF.

Test Case ID	Module	Functionality	Expected Output	Actual Output	Result	Timestamp
TC-EXP-001	Reports & Export	Export all ideas as CSV (admin)	Returns a CSV download	Status 200, type=text/csv; charset=utf-8	Pass	2026-07-28 17:34:49
TC-EXP-002	Reports & Export	Single-idea Closure PDF as a reviewer	Returns a PDF (200)	Status 200, type=application/pdf	Pass	2026-07-28 17:34:50
TC-EXP-003	Reports & Export	Single-idea PDF requested from another tenant	Blocked - tenant boundary	Status 404	Pass	2026-07-28 17:34:50

10.12 Branding & Settings (3/3 passed)

Per-organisation appearance and behaviour, and the rule that the SMTP password is never echoed back once stored.

Test Case ID	Module	Functionality	Expected Output	Actual Output	Result	Timestamp
TC-BRND-001	Branding & Settings	Admin reads branding	Returns branding config	Status 200	Pass	2026-07-28 17:34:50
TC-BRND-002	Branding & Settings	Employee tries to update branding	Forbidden - admin only	Status 403	Pass	2026-07-28 17:34:50
TC-BRND-003	Branding & Settings	Admin saves settings; SMTP secret is never echoed back	Saved; password field masked/omitted on read	Secret not echoed	Pass	2026-07-28 17:34:50

10.13 Support (3/3 passed)

The help desk between a tenant and the vendor, including the two confidentiality rules that make it usable: another organisation cannot read the thread, and a vendor-internal note is never visible to the customer.

Test Case ID	Module	Functionality	Expected Output	Actual Output	Result	Timestamp
TC-SUP-001	Support	User raises a support ticket	Ticket created	Status 200: id=undefined	Pass	2026-07-28 17:34:50
TC-SUP-002	Support	Another org cannot read this org's ticket	Blocked - tenant isolation	Status 404	Pass	2026-07-28 17:34:50
TC-SUP-003	Support	Platform internal note stays hidden from the tenant	Internal message not visible to org users	Internal note hidden	Pass	2026-07-28 17:34:50

10.14 QCMS Integration (7/7 passed)

The outbound path that hands approved ideas to the implementation system. Cases cover saving and masking the API key, overriding the endpoint per organisation, rejecting a malformed endpoint, listing what is pushable, and the administrative boundary on all of it.

Test Case ID	Module	Functionality	Expected Output	Actual Output	Result	Timestamp
TC-QCMS-001	QCMS Integration	Admin saves the QCMS API key	Saved and enabled	Status 200: saved	Pass	2026-07-28 17:34:50
TC-QCMS-002	QCMS Integration	Reading the QCMS config masks the stored key	Key returned masked (****), never in clear	Key masked	Pass	2026-07-28 17:34:50
TC-QCMS-003	QCMS Integration	Admin overrides the QCMS base URL for this tenant	Saved for this tenant; blank restores the environment default	Override saved and cleared	Pass	2026-07-28 17:34:50
TC-QCMS-004	QCMS Integration	A malformed QCMS base URL is rejected	Rejected (400) - ideas are never sent to a bad endpoint	Status 400	Pass	2026-07-28 17:34:50
TC-QCMS-005	QCMS Integration	Non-admin cannot change the QCMS base URL	Forbidden - admin only	Status 403	Pass	2026-07-28 17:34:50
TC-QCMS-006	QCMS Integration	List approved ideas eligible to push (admin)	Returns approved ideas	Status 200	Pass	2026-07-28 17:34:50
TC-QCMS-007	QCMS Integration	Non-admin attempts to push to QCMS	Forbidden - admin only	Status 403	Pass	2026-07-28 17:34:50

10.15 Security & Multi-Tenancy (5/5 passed)

The original isolation cases: content from one organisation must never reach another, a tenant token must not reach vendor routes, injection in a query parameter must be inert, an oversized body must be refused cleanly, and the operational probes must answer.

Test Case ID	Module	Functionality	Expected Output	Actual Output	Result	Timestamp
TC-SEC-001	Security & Multi-Tenancy	Org A user cannot see Org B idea content	Cross-tenant content never exposed (own-tenant record or 404)	No cross-tenant content (status 200)	Pass	2026-07-28 17:34:50
TC-SEC-002	Security & Multi-Tenancy	A tenant JWT cannot reach platform-admin routes	Forbidden	Status 401	Pass	2026-07-28 17:34:50
TC-SEC-003	Security & Multi-Tenancy	SQL injection in a query parameter	Handled safely, no 500/leak	Status 200	Pass	2026-07-28 17:34:50
TC-SEC-004	Security & Multi-Tenancy	Oversized JSON payload	Rejected cleanly (413/400), not a crash	Status 413	Pass	2026-07-28 17:34:50
TC-SEC-005	Security & Multi-Tenancy	Health and readiness probes respond	Liveness ok; readiness reports DB reachability	health=200, ready=200	Pass	2026-07-28 17:34:50

10.16 Safety & Data Protection (40/40 passed)

The deepest module in the suite. It attacks the session (forged, expired, edited, stale, orphaned), attempts privilege escalation directly and by mass assignment, probes file handling from upload through download to deletion, checks that no secret or hash is ever serialised, verifies the browser security headers and the cross-origin policy, confirms that an anonymous submission stays anonymous in every field while remaining accountable in storage, and proves the boot-time guard against insecure production configuration.

Test Case ID	Module	Functionality	Expected Output	Actual Output	Result	Timestamp
TC-SAFE-001	Safety & Data Protection	Unsigned token (alg=none) presented as a session	Rejected 401 - the verifier pins HS256, the token cannot choose	Status 401	Pass	2026-07-28 17:34:50
TC-SAFE-002	Safety & Data Protection	Token signed with an attacker-chosen secret	Rejected 401 - signature must verify against the server secret	Status 401	Pass	2026-07-28 17:34:50
TC-SAFE-003	Safety & Data Protection	Expired session token	Rejected 401 with expired flag, no access	Status 401: Session expired (expired flag)	Pass	2026-07-28 17:34:50
TC-SAFE-004	Safety & Data Protection	Valid token whose role claim was raised to admin	Role is read from the database row, not the token - forbidden	Status 403: Insufficient permissions	Pass	2026-07-28 17:34:50
TC-SAFE-005	Safety & Data Protection	Employee edits their own user record to become admin	Forbidden - role change is admin-only and the stored role is unchanged	Status 403, stored role=employee	Pass	2026-07-28 17:34:50
TC-SAFE-006	Safety & Data Protection	Employee posts role/points through the profile endpoint	Mass-assignment ignored - role and points unchanged	role=employee, points=75 (unchanged)	Pass	2026-07-28 17:34:50

Test Case ID	Module	Functionality	Expected Output	Actual Output	Result	Timestamp
TC-SAFE-007	Safety & Data Protection	Session opened before a password change is reused	Rejected 401 - a password change kills tokens issued earlier	before=200, after password change=401	Pass	2026-07-28 17:34:50
TC-SAFE-008	Safety & Data Protection	Offboarded (deactivated) employee reuses a live token	Rejected 401 on the very next request - status is re-read per request	before=200, after deactivation=401	Pass	2026-07-28 17:34:50
TC-SAFE-009	Safety & Data Protection	Token for an account that has since been deleted	Rejected 401 - no ghost sessions	Status 401: Your account no longer exists.	Pass	2026-07-28 17:34:50
TC-SAFE-010	Safety & Data Protection	User still on a temporary password calls the API directly	Blocked 403 everywhere except the change-password / support allow-list	ideas=403 (You must set a new password before continuing.), me=200	Pass	2026-07-28 17:34:50
TC-SAFE-011	Safety & Data Protection	Password hashes in any authenticated response	Never present - no hash or bcrypt prefix is ever serialised	No hash material in any sampled response	Pass	2026-07-28 17:34:50
TC-SAFE-012	Safety & Data Protection	Browser security headers on an API response	nosniff, frame/CSP lockdown, no-referrer, no server fingerprint	nosniff=nosniff, csp=default-src 'none' ;frame-ancestors 'none', referrer=no-referrer, x-powered-by=absent	Pass	2026-07-28 17:34:50
TC-SAFE-013	Safety & Data Protection	Cross-origin request from a site not on the allow-list	No allow-origin granted to the unlisted site	access-control-allow-origin: absent	Pass	2026-07-28 17:34:50
TC-SAFE-014	Safety & Data Protection	Malformed JSON body on a write endpoint	Clean 400 "Malformed request body", never a 500	Status 400: {"success":false,"error":"Malformed request body."}	Pass	2026-07-28 17:34:50
TC-SAFE-015	Safety & Data Protection	SQL metacharacters stored in an idea title	Stored as literal text - the ideas table is untouched	ideas rows before=5, after=6 (table intact)	Pass	2026-07-28 17:34:50
TC-SAFE-016	Safety & Data Protection	Stored HTML/script payload read back through the API	Returned as JSON data, never as executable HTML	content-type=application/json; charset=utf-8	Pass	2026-07-28 17:34:50
TC-SAFE-017	Safety & Data Protection	Attachment with a disallowed extension (.exe)	Rejected 400 - extension allow-list enforced server-side	Status 400: File type not allowed.	Pass	2026-07-28 17:34:50
TC-SAFE-018	Safety & Data Protection	Attachment with a double extension (report.pdf.exe)	Rejected - the real (last) extension is what is checked	Status 400: File type not allowed.	Pass	2026-07-28 17:34:50
TC-SAFE-019	Safety & Data Protection	Attachment over the configured size limit	Clean 400 stating the limit, not a dropped connection	Status 400: File exceeds 10MB limit.	Pass	2026-07-28 17:34:51
TC-SAFE-020	Safety & Data Protection	Employee attaches a valid file to their own idea	Accepted and recorded against that idea only	Status 200, attachment id=1	Pass	2026-07-28 17:34:51

Test Case ID	Module	Functionality	Expected Output	Actual Output	Result	Timestamp
TC-SAFE-021	Safety & Data Protection	Attaching a file to someone else's idea	Forbidden - ownership checked before the file is written	Status 403: Unauthorized or idea not found.	Pass	2026-07-28 17:34:51
TC-SAFE-022	Safety & Data Protection	Downloading an attachment with no session	Rejected 401 - attachments are not public URLs	Status 401	Pass	2026-07-28 17:34:51
TC-SAFE-023	Safety & Data Protection	Downloading another organisation's attachment id	Not found - the lookup is scoped to the caller's tenant database	Status 404: Attachment not found.	Pass	2026-07-28 17:34:51
TC-SAFE-024	Safety & Data Protection	Legitimate attachment download by a permitted user	Streamed as an attachment with nosniff - never renderable in-origin	Status 200, disposition=attachment; filename="evidence.png"; file, nosniff=nosniff	Pass	2026-07-28 17:34:51
TC-SAFE-025	Safety & Data Protection	Path traversal in the attachment id	Rejected - id is numeric, no filesystem path is ever taken from input	Status 400	Pass	2026-07-28 17:34:51
TC-SAFE-026	Safety & Data Protection	Deleting an attachment belonging to another user	Forbidden - only the owner may remove their file	Status 403: Not found or unauthorized.	Pass	2026-07-28 17:34:51
TC-SAFE-027	Safety & Data Protection	Anonymous idea opened by a colleague	Submitter name, e-mail and department masked from the peer	Identity masked for the colleague	Pass	2026-07-28 17:34:51
TC-SAFE-028	Safety & Data Protection	Anonymous idea opened by its own author	Author still sees their own submission unmasked	Author sees their own name	Pass	2026-07-28 17:34:51
TC-SAFE-029	Safety & Data Protection	Anonymous idea still auditable in storage	submitter_id retained in the database for accountability	submitter_id=2, is_anonymous=1	Pass	2026-07-28 17:34:51
TC-SAFE-030	Safety & Data Protection	Platform (vendor) admin reads a customer's ideas	Refused - vendor staff see counts, never tenant content	ideas=500, users=500, no tenant content	Pass	2026-07-28 17:34:51
TC-SAFE-031	Safety & Data Protection	Login to a suspended organisation	Blocked while suspended, restored on reactivation	suspended: 404 (Unknown organization code.); reactivated: login ok	Pass	2026-07-28 17:34:51
TC-SAFE-032	Safety & Data Protection	Five wrong passwords then the CORRECT one	Account locked out - the right password does not clear a live lockout	Blocked: Too many failed attempts. Please try again in 15 minute(s).	Pass	2026-07-28 17:34:51
TC-SAFE-033	Safety & Data Protection	Lockout counter is stored centrally, not per process	Recorded in the master registry so every instance sees it	1 row(s) persisted, attempts=1	Pass	2026-07-28 17:34:51
TC-SAFE-034	Safety & Data Protection	Server error body inspected for internals	Generic message only - no stack trace, SQL or file path	Clean 404: {"success":false,"error":"Idea not found"}	Pass	2026-07-28 17:34:51

Test Case ID	Module	Functionality	Expected Output	Actual Output	Result	Timestamp
TC-SAFE-035	Safety & Data Protection	Request to an endpoint that does not exist	Uniform 404 envelope, no framework fingerprint	Status 404: {"success":false,"error":"Unknown action"}	Pass	2026-07-28 17:34:51
TC-SAFE-036	Safety & Data Protection	Rate-limit budget is advertised to clients	Standard RateLimit headers present on API responses	RateLimit headers present (budget shown: 1000000; the suite raises GLOBAL_RATE_LIMIT - the shipped default is 300/min per IP, 30/15min on auth)	Pass	2026-07-28 17:34:51
TC-SAFE-037	Safety & Data Protection	Employee reads the integration (API key) screen	Forbidden - secrets are admin-only	Status 403	Pass	2026-07-28 17:34:51
TC-SAFE-038	Safety & Data Protection	Production configuration guard against insecure defaults	Placeholder JWT secret / rootless DB flagged before boot	6 problem(s) reported, first: JWT_SECRET is unset or still the example placeholder. Anyone with the repo can forge auth	Pass	2026-07-28 17:34:51
TC-SAFE-039	Safety & Data Protection	Password shorter than the policy minimum	Rejected - minimum length enforced server-side	Status 400: Password must be at least 12 characters.	Pass	2026-07-28 17:34:51
TC-SAFE-040	Safety & Data Protection	CRLF injection through a user-supplied name	Stored as text - no header split, no 500	Status 200	Pass	2026-07-28 17:34:51

10.17 Reliability & Fault Tolerance (24/24 passed)

What happens when input is malformed, when several things happen at once, and when a dependency is broken. The concurrency cases here found two real defects in this cycle, both since fixed and both now permanently guarded.

Test Case ID	Module	Functionality	Expected Output	Actual Output	Result	Timestamp
TC-REL-001	Reliability & Fault Tolerance	Request body sent as text/plain to a JSON endpoint	Handled as a client error, never a 500	Status 400	Pass	2026-07-28 17:34:51
TC-REL-002	Reliability & Fault Tolerance	POST with no body at all	Validation error, service stays up	Status 400: Title, present situation and proposed solution are required.	Pass	2026-07-28 17:34:51
TC-REL-003	Reliability & Fault Tolerance	Null values in required fields	Validation error, not a null-dereference crash	Status 400: Title, present situation and proposed solution are required.	Pass	2026-07-28 17:34:51
TC-REL-004	Reliability & Fault Tolerance	JSON array sent where an object is expected	Rejected cleanly	Status 400	Pass	2026-07-28 17:34:51

Test Case ID	Module	Functionality	Expected Output	Actual Output	Result	Timestamp
TC-REL-005	Reliability & Fault Tolerance	Deeply nested JSON (200 levels)	Parsed or rejected without exhausting the stack	Status 400	Pass	2026-07-28 17:34:51
TC-REL-006	Reliability & Fault Tolerance	Absurd numeric value in a money field	Handled without overflow or crash	Status 200: accepted	Pass	2026-07-28 17:34:51
TC-REL-007	Reliability & Fault Tolerance	Non-numeric id in a path parameter	Client error, not a 500	Status 404	Pass	2026-07-28 17:34:51
TC-REL-008	Reliability & Fault Tolerance	Ten simultaneous votes from one user on one idea	At most one stored vote - no double counting under concurrency	Stored vote rows: 0	Pass	2026-07-28 17:34:51
TC-REL-009	Reliability & Fault Tolerance	Ten simultaneous comments on one idea	All accepted, none lost, no deadlock	10/10 stored, 0 server errors	Pass	2026-07-28 17:34:51
TC-REL-010	Reliability & Fault Tolerance	Five simultaneous approvals of the same idea	Exactly one approval recorded - the guard holds under a race	Approved workflow rows: 1	Pass	2026-07-28 17:34:51
TC-REL-011	Reliability & Fault Tolerance	Two simultaneous account creations with the same email	Only one account exists afterwards	Accounts with that email: 1	Pass	2026-07-28 17:34:51
TC-REL-012	Reliability & Fault Tolerance	QCMS endpoint refuses connections	Push reports failed per idea; the platform keeps serving	push status=200, failed=1, health=200	Pass	2026-07-28 17:34:52
TC-REL-013	Reliability & Fault Tolerance	QCMS accepts the connection then never responds	Timed out and classified as failed, no hung request	status=failed in 807 ms: QCMS did not respond in time.	Pass	2026-07-28 17:34:52
TC-REL-014	Reliability & Fault Tolerance	Idea submitted while the mail server is unreachable	Idea is still saved - notification failure never blocks the user	Status 200, stored=1	Pass	2026-07-28 17:34:52
TC-REL-015	Reliability & Fault Tolerance	Approval recorded while the mail server is unreachable	Decision persists even though the e-mail cannot be sent	Status 200, stored status=Approved	Pass	2026-07-28 17:34:52
TC-REL-016	Reliability & Fault Tolerance	Sixty mixed requests fired back to back	No 5xx across the burst; the service stays healthy	60/60 clean, health afterwards=200	Pass	2026-07-28 17:34:53
TC-REL-017	Reliability & Fault Tolerance	Multilingual and emoji content round-trip	Stored and returned byte-identical (utf8mb4 end to end)	Exact round-trip	Pass	2026-07-28 17:34:53
TC-REL-018	Reliability & Fault Tolerance	Comment containing 4-byte emoji	Accepted and stored without mangling	Status 200, stored: Good catch ???	Pass	2026-07-28 17:34:53
TC-REL-019	Reliability & Fault Tolerance	Marking notifications read twice	Idempotent - the second call is still a success	first=200, second=200	Pass	2026-07-28 17:34:53

Test Case ID	Module	Functionality	Expected Output	Actual Output	Result	Timestamp
TC-REL-020	Reliability & Fault Tolerance	Whitespace-only title	Rejected - trimmed validation, no blank ideas	Status 400: Title, present situation and proposed solution are required.	Pass	2026-07-28 17:34:53
TC-REL-021	Reliability & Fault Tolerance	Near-duplicate idea title check	Similar existing ideas are surfaced before submission	Status 200, 0 similar idea(s) reported	Pass	2026-07-28 17:34:53
TC-REL-022	Reliability & Fault Tolerance	60,000-character solution text	Accepted and returned intact, or rejected cleanly - never a 500	Status 200, 60000 chars stored	Pass	2026-07-28 17:34:53
TC-REL-023	Reliability & Fault Tolerance	Ten login/logout cycles in a row	Every cycle succeeds - no session leak or degradation	10/10 cycles clean	Pass	2026-07-28 17:34:53
TC-REL-024	Reliability & Fault Tolerance	Export requested while writes are in flight	Export completes; concurrent writes are unaffected	export=200, 5/5 writes stored	Pass	2026-07-28 17:34:53

10.18 Scalability - Horizontal (16/16 passed)

A second application process against the same databases - the shape of a load-balanced deployment. Sessions, data, settings, branding, integration credentials, new tenants and the brute-force lockout must all behave identically whichever instance a request reaches, and killing one instance must not disturb the other.

Test Case ID	Module	Functionality	Expected Output	Actual Output	Result	Timestamp
TC-SCLH-001	Scalability - Horizontal	A second application instance starts against the same databases	Instance 2 boots and answers its liveness probe	Instance 2 on http://127.0.0.1:55460, health=200	Pass	2026-07-28 17:34:55
TC-SCLH-002	Scalability - Horizontal	Readiness probe on the second instance	Reports ready - the load balancer may route to it	Status 200: ready	Pass	2026-07-28 17:34:55
TC-SCLH-003	Scalability - Horizontal	Session issued by instance 1 used on instance 2	Accepted - authentication is stateless, no sticky sessions needed	Status 200	Pass	2026-07-28 17:34:55
TC-SCLH-004	Scalability - Horizontal	Session issued by instance 2 used on instance 1	Accepted in both directions	login on 2 = ok, request on 1 = 200	Pass	2026-07-28 17:34:55
TC-SCLH-005	Scalability - Horizontal	Idea created on instance 1 read from instance 2	Immediately visible - no per-process cache to go stale	Status 200	Pass	2026-07-28 17:34:55
TC-SCLH-006	Scalability - Horizontal	Idea created on instance 2 read from instance 1	Immediately visible in the other direction	Status 200	Pass	2026-07-28 17:34:55
TC-SCLH-007	Scalability - Horizontal	Vote cast on instance 2, counted once on instance 1	One vote - shared state lives in the database, not in a process	Stored votes: 1	Pass	2026-07-28 17:34:55

Test Case ID	Module	Functionality	Expected Output	Actual Output	Result	Timestamp
TC-SCLH-008	Scalability - Horizontal	Organisation settings changed on 1 take effect on 2	No configuration cached per process	Instance 2 reports review_sla_days=11	Pass	2026-07-28 17:34:56
TC-SCLH-009	Scalability - Horizontal	Integration credentials saved on 1 are usable on 2	Per-tenant integration config is shared, not per instance	Instance 2 reports base_url=https://qcms.shared.example/v1	Pass	2026-07-28 17:34:56
TC-SCLH-010	Scalability - Horizontal	Branding updated on 1 is served by 2	Tenant branding is read from the database on both	Instance 2 branding: {"success":true,"branding":{"org_name":"Org A Renamed","logo":null,"logo_updated	Pass	2026-07-28 17:34:56
TC-SCLH-011	Scalability - Horizontal	Failed logins counted on 1 lock the account on 2	Brute-force state is central - an attacker cannot rotate instances	Instance 2 also blocked: Too many failed attempts. Please try again in 15 minute(s).	Pass	2026-07-28 17:34:56
TC-SCLH-012	Scalability - Horizontal	Organisation created on 1 is served by 2 with no restart	New tenants are picked up from the registry at request time	create=200, login on instance 2=ok	Pass	2026-07-28 17:34:58
TC-SCLH-013	Scalability - Horizontal	Twenty requests alternating between the two instances	All succeed - any request may land on any instance	20/20 succeeded across both instances	Pass	2026-07-28 17:34:58
TC-SCLH-014	Scalability - Horizontal	Attachment uploaded through 1, downloaded through 2	Served by either instance when the upload directory is shared storage	Status 200 - note: requires shared/NFS storage or object storage across hosts	Pass	2026-07-28 17:34:58
TC-SCLH-015	Scalability - Horizontal	Per-IP rate-limit counters across two instances	Counters are per process - a strict global cap needs a shared store	Instance 2 unaffected by instance 1 traffic (health=200); documented limitation - use a Redis store for one global budget	Pass	2026-07-28 17:34:58
TC-SCLH-016	Scalability - Horizontal	One instance is killed mid-service	The surviving instance keeps serving - no shared in-process state	instance 1 = 200 after instance 2 was killed (instance 2 = down)	Pass	2026-07-28 17:34:59

10.19 Scalability - Vertical & Performance (21/21 passed)

The same machine holding far more data. One organisation is loaded with five thousand ideas and three hundred users, and then every screen is held to a latency budget, the response cap is proven to be server-side, the query plan is inspected, concurrency is pushed past the connection pool and the neighbouring organisation is checked for collateral damage.

Test Case ID	Module	Functionality	Expected Output	Actual Output	Result	Timestamp
TC-SCLV-001	Scalability - Vertical & Performance	Load one organisation with 5,000 ideas and 300 users	Dataset lands in the tenant database and is queryable	5023 ideas in 1183 ms (4227 rows/s)	Pass	2026-07-28 17:35:00

Test Case ID	Module	Functionality	Expected Output	Actual Output	Result	Timestamp
TC-SCLV-002	Scalability - Vertical & Performance	Idea list response size at 5,000 ideas	Bounded at 100 rows - response size does not grow with the dataset	100 rows returned, payload 200 KB	Pass	2026-07-28 17:35:00
TC-SCLV-003	Scalability - Vertical & Performance	Client attempts to raise the row cap (?limit=100000)	Cap is server-side - a client cannot ask for the whole table	100 rows returned	Pass	2026-07-28 17:35:00
TC-SCLV-004	Scalability - Vertical & Performance	Idea list latency under load	Responds in under 3000 ms at 5,000 ideas	Status 200 in 79 ms (budget 3000 ms)	Pass	2026-07-28 17:35:00
TC-SCLV-005	Scalability - Vertical & Performance	Dashboard aggregate latency under load	Responds in under 3000 ms at 5,000 ideas	Status 200 in 21 ms (budget 3000 ms)	Pass	2026-07-28 17:35:00
TC-SCLV-006	Scalability - Vertical & Performance	Analytics latency under load	Responds in under 4000 ms at 5,000 ideas	Status 200 in 33 ms (budget 4000 ms)	Pass	2026-07-28 17:35:00
TC-SCLV-007	Scalability - Vertical & Performance	Leaderboard latency with 300+ users	Responds in under 3000 ms at 5,000 ideas	Status 200 in 60 ms (budget 3000 ms)	Pass	2026-07-28 17:35:00
TC-SCLV-008	Scalability - Vertical & Performance	Audit report latency under load	Responds in under 4000 ms at 5,000 ideas	Status 200 in 7 ms (budget 4000 ms)	Pass	2026-07-28 17:35:00
TC-SCLV-009	Scalability - Vertical & Performance	Title search latency under load	Responds in under 3000 ms at 5,000 ideas	Status 200 in 86 ms (budget 3000 ms)	Pass	2026-07-28 17:35:00
TC-SCLV-010	Scalability - Vertical & Performance	Status-filtered list latency under load	Responds in under 3000 ms at 5,000 ideas	Status 200 in 26 ms (budget 3000 ms)	Pass	2026-07-28 17:35:00
TC-SCLV-011	Scalability - Vertical & Performance	Approved-ideas (integration) listing under load	Responds in under 4000 ms at 5,000 ideas	Status 200 in 36 ms (budget 4000 ms)	Pass	2026-07-28 17:35:00
TC-SCLV-012	Scalability - Vertical & Performance	Single idea detail stays constant-time	Responds in under 1000 ms at 5,000 ideas	Status 200 in 26 ms (budget 1000 ms)	Pass	2026-07-28 17:35:00
TC-SCLV-013	Scalability - Vertical & Performance	Employee's own idea list under load	Responds in under 3000 ms at 5,000 ideas	Status 200 in 254 ms (budget 3000 ms)	Pass	2026-07-28 17:35:01
TC-SCLV-014	Scalability - Vertical & Performance	Index behind the list ordering exists	idx_ideas_updated_at present so the top-100 read is an ordered index scan	Index present on column updated_at	Pass	2026-07-28 17:35:01
TC-SCLV-015	Scalability - Vertical & Performance	Query plan for the list query at 5,000 rows	No full scan of ideas with a filesort of the entire table	type=index, key=idx_ideas_updated_at, rows=100, extra=-	Pass	2026-07-28 17:35:01
TC-SCLV-016	Scalability - Vertical & Performance	CSV export of the full 5,000-idea dataset	Completes inside 15 s and contains every row	5024 lines, 0.6 MB in 55 ms	Pass	2026-07-28 17:35:01

Test Case ID	Module	Functionality	Expected Output	Actual Output	Result	Timestamp
TC-SCLV-017	Scalability - Vertical & Performance	Thirty concurrent requests against a pool of 10	All served - requests queue for a connection instead of failing	30/30 succeeded in 201 ms with DB_POOL_SIZE=10	Pass	2026-07-28 17:35:01
TC-SCLV-018	Scalability - Vertical & Performance	Sustained throughput on the loaded dataset	One hundred reads with no errors, throughput recorded	100 requests in 1805 ms -> 55.4 req/s, 0 errors	Pass	2026-07-28 17:35:03
TC-SCLV-019	Scalability - Vertical & Performance	The other organisation while this one holds 5,000 ideas	Unaffected - each tenant is a separate schema with its own indexes	Org B dashboard 200 in 13 ms	Pass	2026-07-28 17:35:03
TC-SCLV-020	Scalability - Vertical & Performance	Process memory after the load run	Resident memory stays bounded - no dataset is held in the application	RSS 180 MB, heap 87 MB after 5,000 ideas	Pass	2026-07-28 17:35:03
TC-SCLV-021	Scalability - Vertical & Performance	Notification list is bounded	Returns a capped page rather than every notification ever raised	3 notifications returned	Pass	2026-07-28 17:35:03

10.20 Data Integrity & Recovery (17/17 passed)

The properties that decide whether the data can be trusted and rebuilt: an idempotent, ledgered migration runner that picks up a brand-new file, unique constraints, cascade behaviour, the atomicity of a decision, points arithmetic, timestamp consistency, character set, storage engine, primary keys, and the backup and restore path.

Test Case ID	Module	Functionality	Expected Output	Actual Output	Result	Timestamp
TC-DATA-001	Data Integrity & Recovery	Migration runner applied against the live schemas	Pending migrations applied and recorded in the ledger	30 application(s) this pass, 30 ledger row(s) total	Pass	2026-07-28 17:35:03
TC-DATA-002	Data Integrity & Recovery	Migration runner executed a second time	Applies nothing - migrations are idempotent and ledgered	0 application(s) on the repeat run	Pass	2026-07-28 17:35:03
TC-DATA-003	Data Integrity & Recovery	A brand-new migration file is dropped into the folder	Picked up automatically on the next run, then recorded	4 application(s); probe table present in tenant A = true	Pass	2026-07-28 17:35:03
TC-DATA-004	Data Integrity & Recovery	Ledger records which schema received which migration	One row per (database, migration) - reproducible on a new environment	ifqm_acme:7, ifqm_growth:7, ifqm_test_a:7, ifqm_test_b:7, ifqm_test_master:2	Pass	2026-07-28 17:35:03
TC-DATA-005	Data Integrity & Recovery	A tenant provisioned by the product versus the reference schema	Same table set - a new organisation is not a second-class database	reference 18 tables, provisioned 19, missing: none	Pass	2026-07-28 17:35:03

Test Case ID	Module	Functionality	Expected Output	Actual Output	Result	Timestamp
TC-DATA-006	Data Integrity & Recovery	Duplicate idea code inserted directly	Rejected by the UNIQUE constraint - codes cannot collide	Rejected: ER_DUP_ENTRY	Pass	2026-07-28 17:35:03
TC-DATA-007	Data Integrity & Recovery	Idea deleted while it still has children	Comments, votes, attachments and workflow rows go with it	orphans left - comments:0, votes:0, workflow:0	Pass	2026-07-28 17:35:04
TC-DATA-008	Data Integrity & Recovery	Idea code sequence after a deletion	Next code continues past the highest issued - no reuse, no collision	previous max 23 -> issued IDA-2026-024	Pass	2026-07-28 17:35:04
TC-DATA-009	Data Integrity & Recovery	Approval writes the status and the audit entry together	Both present - a decision is never half-recorded	status=Approved, workflow rows=2	Pass	2026-07-28 17:35:04
TC-DATA-010	Data Integrity & Recovery	Points awarded on submission match the configured value	Ledger and configuration agree	+10 points (POINTS_SUBMIT=10)	Pass	2026-07-28 17:35:04
TC-DATA-011	Data Integrity & Recovery	Stored timestamps versus the database clock	Written in the database's own time - no timezone drift	created_at is 0 s from NOW()	Pass	2026-07-28 17:35:04
TC-DATA-012	Data Integrity & Recovery	Character set of the tenant tables	utf8mb4 throughout - every language and emoji storable	0 table(s) not on utf8mb4	Pass	2026-07-28 17:35:04
TC-DATA-013	Data Integrity & Recovery	Storage engine of the tenant tables	InnoDB throughout - transactional and crash-recoverable	0 table(s) not InnoDB	Pass	2026-07-28 17:35:04
TC-DATA-014	Data Integrity & Recovery	Primary keys on every tenant table	Required for replication and for safe row-level recovery	Every table has a primary key	Pass	2026-07-28 17:35:04
TC-DATA-015	Data Integrity & Recovery	Backup script is parameterised, not hard-coded	Reads credentials and schema names from configuration	env-driven credentials=true, schemas discovered from the registry=false, hard-coded secrets=false	Pass	2026-07-28 17:35:04
TC-DATA-016	Data Integrity & Recovery	Restore procedure documented for operators	Deployment guide covers backup and restore	backup mentioned=true, restore mentioned=true	Pass	2026-07-28 17:35:04
TC-DATA-017	Data Integrity & Recovery	Suspended organisation reactivated	All of its data is intact after the outage	ideas before=1, after=1	Pass	2026-07-28 17:35:04

10.21 Extensibility & Future Scope (18/18 passed)

Evidence that the next requirement is an addition rather than a rewrite: a tenant onboarded at runtime, two organisations on different integration endpoints simultaneously, every limit read from configuration, complete translation coverage across seven languages, a new migration applied, and a pure mapping function for the

downstream system.

Test Case ID	Module	Functionality	Expected Output	Actual Output	Result	Timestamp
TC-FUT-001	Extensibility & Future Scope	Translation coverage across all shipped languages	Every locale carries the same key set as English	en:799 - hi:799 - kn:799 - ml:799 - mr:799 - ta:799 - te:799	Pass	2026-07-28 17:35:04
TC-FUT-002	Extensibility & Future Scope	Adding an eighth language	Purely additive - one dictionary file plus one registry line	All seven locales registered in one map - an eighth is one import + one entry	Pass	2026-07-28 17:35:04
TC-FUT-003	Extensibility & Future Scope	Adding a new API area	Feature modules are mounted in one aggregator, no core surgery	18 feature modules mounted under /api	Pass	2026-07-28 17:35:04
TC-FUT-004	Extensibility & Future Scope	Adding a new reviewer role	Roles are data in one list per route file, not scattered conditionals	Reviewer and implementer role sets are single declarations	Pass	2026-07-28 17:35:04
TC-FUT-005	Extensibility & Future Scope	Onboarding a new organisation at runtime	Usable the moment it is created - no deploy, no restart	login=ok, dashboard=200	Pass	2026-07-28 17:35:04
TC-FUT-006	Extensibility & Future Scope	Two organisations holding different integration endpoints	Per-tenant configuration - customers can point at their own systems	org A -> https://qcms-a.example/v1, org B -> https://qcms-b.example/v2	Pass	2026-07-28 17:35:05
TC-FUT-007	Extensibility & Future Scope	Capacity and policy knobs without a code change	Limits, pool size, token life and points all come from the environment	GLOBAL_RATE_LIMIT=1000000, AUTH_RATE_LIMIT=10000, DB_POOL_SIZE=10, MAX_FILE_MB=10, JWT_EXPIRES_IN=28800, POINTS_SUBMIT=10, POINTS_APPROVED=25, POINTS_IMPLEMENTED=65, MIN_PASSWORD_LENGTH=12	Pass	2026-07-28 17:35:05
TC-FUT-008	Extensibility & Future Scope	Tenants can be spread across database servers	The registry stores a host per tenant, so growth means another DB box	orga@localhost/ifqm_test_a - orgb@localhost/ifqm_test_b - acme@localhost/ifqm_acme - growth@localhost/ifqm_growth	Pass	2026-07-28 17:35:05
TC-FUT-009	Extensibility & Future Scope	AI scoring with no provider key configured	Falls back to the built-in heuristic - the feature never hard-fails	Status 200, provider="none - heuristic"	Pass	2026-07-28 17:35:05
TC-FUT-010	Extensibility & Future Scope	Swapping the AI provider	Provider and keys are configuration, not embedded calls	config.ai exposes: provider, openaiApiKey, geminiApiKey	Pass	2026-07-28 17:35:05

Test Case ID	Module	Functionality	Expected Output	Actual Output	Result	Timestamp
TC-FUT-011	Extensibility & Future Scope	Targeting a second downstream system	The QCMS field mapping is a pure function - a new target is a new mapper	mapped 11 fields with no I/O: category=Cost, status=Approved	Pass	2026-07-28 17:35:05
TC-FUT-012	Extensibility & Future Scope	Response envelope is uniform across features	New clients can rely on one success/error shape	5/5 endpoints return the standard envelope	Pass	2026-07-28 17:35:05
TC-FUT-013	Extensibility & Future Scope	Continuous integration runs this suite	Every push is gated by the automated tests	ci.yml runs npm test on push	Pass	2026-07-28 17:35:05
TC-FUT-014	Extensibility & Future Scope	Runtime floor is declared	package.json pins the supported Node version for future upgrades	engines.node = >=18	Pass	2026-07-28 17:35:05
TC-FUT-015	Extensibility & Future Scope	Reproducible dependency installs	Lockfile committed so a future build resolves the same tree	backend/package-lock.json present	Pass	2026-07-28 17:35:05
TC-FUT-016	Extensibility & Future Scope	No production database name hard-coded in the source	Schema names come from configuration, so environments stay separable	referenced in: src\config\index.js	Pass	2026-07-28 17:35:05
TC-FUT-017	Extensibility & Future Scope	Operator and end-user documentation ships with the code	Deployment, user guide and changelog are in the repository	4 documents present	Pass	2026-07-28 17:35:05
TC-FUT-018	Extensibility & Future Scope	Front end builds to static assets	Can be served from a CDN or any number of web nodes	build script: vite build	Pass	2026-07-28 17:35:05

10.22 Observability & Operations (8/8 passed)

What the team running this in production depends on: cheap liveness, meaningful readiness, graceful shutdown, crash handlers, a durable log trail, an audit report and a configuration guard at boot.

Test Case ID	Module	Functionality	Expected Output	Actual Output	Result	Timestamp
TC-OPS-001	Observability & Operations	Liveness probe cost	Answers in single-digit milliseconds - safe to poll often	Status 200 in 2 ms	Pass	2026-07-28 17:35:05
TC-OPS-002	Observability & Operations	Readiness probe reports database reachability	Distinct from liveness so a database-less instance leaves rotation	Status 200: {"success":true,"status":"ready"}	Pass	2026-07-28 17:35:05
TC-OPS-003	Observability & Operations	Graceful shutdown is wired up	SIGTERM/SIGINT drain in-flight requests and close pools	Drain-then-close handler registered for SIGINT and SIGTERM	Pass	2026-07-28 17:35:05

Test Case ID	Module	Functionality	Expected Output	Actual Output	Result	Timestamp
TC-OPS-004	Observability & Operations	Crash handlers registered	Unhandled rejections and exceptions are logged, not silent	unhandledRejection and uncaughtException handlers present	Pass	2026-07-28 17:35:05
TC-OPS-005	Observability & Operations	Durable log trail for support	Daily files, location configurable, enabled in production	LOG_TO_FILE / LOG_DIR honoured; separate daily error log	Pass	2026-07-28 17:35:05
TC-OPS-006	Observability & Operations	Audit trail available to an administrator	Who did what, retrievable from the product itself	Status 200, 31 entries returned	Pass	2026-07-28 17:35:05
TC-OPS-007	Observability & Operations	Configuration is validated at boot	Insecure production settings stop the process rather than run wide open	server.js calls assertConfigOrExit before listening	Pass	2026-07-28 17:35:05
TC-OPS-008	Observability & Operations	Release history is maintained	CHANGELOG present for operators tracking what shipped	141 lines of change history	Pass	2026-07-28 17:35:05

11. Appendix A - Suite inventory

The files that produced this document, and what each is responsible for.

backend/test/tc_runner.mjs	The runner. Boots the application, drives every case over HTTP, records the seven fields per case and writes tc_results.json.
backend/test/helpers.js	Scratch-environment harness: creates and drops the registry and tenant schemas, seeds the accounts, starts the application, exposes the HTTP and SQL helpers.
backend/test/instance2.mjs	The second application process used by the horizontal-scalability module.
backend/test/api.test.js	The assertion suite run by "npm test" - 33 tests that gate every push in CI.
backend/test/tc_results.json	The machine-written evidence file this document is generated from.
docs/gen_testcases_doc.py	This generator.

12. Appendix B - Endpoint inventory and access rules

Generated by reading the route definitions in backend/src/routes at build time, so it cannot drift from the code. "Guard" is the access rule the route declares; every guard also re-reads the caller's role from the database before it is applied.

Method and path	Guard	Module
GET /api/auth/me	Open (answers signed-in and signed-out)	Auth
POST /api/auth/login	Open, rate limited	Auth
POST /api/auth/logout	Open	Auth
POST /api/auth/forgot-password	Open, rate limited	Auth
POST /api/auth/reset-password	Open, rate limited	Auth
GET /api/auth/check-reset-token	Open	Auth
POST /api/auth/change-password	Any signed-in user	Auth
GET /api/branding	Any signed-in user	Branding
PUT /api/branding	Roles: admin, super_admin	Branding
POST /api/branding/logo	Roles: admin, super_admin	Branding
DELETE /api/branding/logo	Roles: admin, super_admin	Branding
GET /api/categories	Any signed-in user	Categories
POST /api/categories	Roles: admin, super_admin	Categories
DELETE /api/categories/:id	Roles: admin, super_admin	Categories
GET /api/challenges	Any signed-in user	Challenges
GET /api/challenges/:id	Any signed-in user	Challenges
POST /api/challenges	Declared role list	Challenges
PUT /api/challenges/:id	Declared role list	Challenges
DELETE /api/challenges/:id	Roles: admin, super_admin	Challenges
GET /api/comments	Any signed-in user	Comments
POST /api/comments	Any signed-in user	Comments
DELETE /api/comments/:id	Any signed-in user	Comments
GET /api/export/ideas	Any signed-in user	Export
GET /api/export/leaderboard	Any signed-in user	Export
GET /api/export/analytics	Analytics roles (manager and above)	Export
GET /api/export/idea/:id/pdf	Reviewer roles (team lead and above)	Export
GET /api/ideas	Any signed-in user	Ideas
GET /api/ideas/my	Any signed-in user	Ideas
GET /api/ideas/review	Reviewer roles (team lead and above)	Ideas
GET /api/ideas/dashboard	Any signed-in user	Ideas
GET /api/ideas/check-duplicate	Any signed-in user	Ideas
GET /api/ideas/:id	Any signed-in user	Ideas

Method and path	Guard	Module
POST /api/ideas/submit	Any signed-in user	Ideas
POST /api/ideas/draft	Any signed-in user	Ideas
POST /api/ideas/review-action	Reviewer roles (team lead and above)	Ideas
POST /api/ideas/assign-reviewers	Reviewer roles (team lead and above)	Ideas
POST /api/ideas/reviewer-decision	Reviewer roles (team lead and above)	Ideas
POST /api/ideas/bulk-review	Reviewer roles (team lead and above)	Ideas
POST /api/ideas/roi	Implementer roles (manager and above)	Ideas
POST /api/ideas/implementation	Implementer roles (manager and above)	Ideas
GET /api/integrations/approved-ideas	Declared role list	Integrations
GET /api/integrations/qcms	Declared role list	Integrations
PUT /api/integrations/qcms	Declared role list	Integrations
POST /api/integrations/push	Declared role list	Integrations
GET /api/leaderboard	Any signed-in user	Leaderboard
GET /api/notifications	Any signed-in user	Notifications
POST /api/notifications/mark-read	Any signed-in user	Notifications
GET /api/platform/tenants	Platform (vendor) admin	Platform
GET /api/platform/tenants/:id	Platform (vendor) admin	Platform
POST /api/platform/tenants	Platform (vendor) admin	Platform
PATCH /api/platform/tenants/:id	Platform (vendor) admin	Platform
POST /api/platform/tenants/:id/reset-admin-password	Platform (vendor) admin	Platform
DELETE /api/platform/tenants/:id	Platform (vendor) admin	Platform
GET /api/platform/settings/defaults	Platform (vendor) admin	Platform
PUT /api/platform/settings/defaults	Platform (vendor) admin	Platform
GET /api/platform/tenants/:id/settings	Platform (vendor) admin	Platform
PUT /api/platform/tenants/:id/settings	Platform (vendor) admin	Platform
GET /api/platform/admins	Platform (vendor) admin	Platform
POST /api/platform/admins	Platform (vendor) admin	Platform
DELETE /api/platform/admins/:id	Platform (vendor) admin	Platform
POST /api/platform/admins/change-password	Platform (vendor) admin	Platform
GET /api/platform/health	Platform (vendor) admin	Platform
GET /api/platform/tickets	Platform (vendor) admin	Platform
POST /api/platform/tickets	Platform (vendor) admin	Platform
GET /api/platform/tickets/:id	Platform (vendor) admin	Platform
POST /api/platform/tickets/:id/messages	Platform (vendor) admin	Platform
PATCH /api/platform/tickets/:id	Platform (vendor) admin	Platform

Method and path	Guard	Module
GET /api/reports/analytics	Roles: admin, executive, manager, department_manager, senior_manager, plant_head, super_admin	Reports
GET /api/reports/audit	Roles: admin, manager, department_manager, senior_manager, plant_head, executive, super_admin	Reports
GET /api/score	Any signed-in user	Score
POST /api/score/batch-rescore	Roles: admin	Score
GET /api/settings	Any signed-in user	Settings
POST /api/settings	Roles: admin, super_admin	Settings
GET /api/settings/test-email	Roles: admin, super_admin	Settings
GET /api/support/tickets	Any signed-in user	Support
POST /api/support/tickets	Any signed-in user	Support
GET /api/support/tickets/:id	Any signed-in user	Support
POST /api/support/tickets/:id/messages	Any signed-in user	Support
PATCH /api/support/tickets/:id	Any signed-in user	Support
POST /api/upload	Any signed-in user	Upload
GET /api/upload/:id/download	Any signed-in user	Upload
DELETE /api/upload/:id	Any signed-in user	Upload
GET /api/users/import/template	Organisation admin	Users
POST /api/users/import/preview	Organisation admin	Users
POST /api/users/import	Organisation admin	Users
GET /api/users/import/:id	Organisation admin	Users
GET /api/users/import/:id/errors.csv	Organisation admin	Users
GET /api/users	Any signed-in user	Users
GET /api/users/admin	Roles: admin, super_admin	Users
GET /api/users/managers	Roles: admin, super_admin	Users
GET /api/users/hierarchy	Roles: admin, super_admin	Users
POST /api/users/profile	Any signed-in user	Users
POST /api/users	Roles: admin, super_admin	Users
PUT /api/users/:id/manager	Roles: admin, super_admin	Users
PUT /api/users/:id	Roles: admin, super_admin	Users
DELETE /api/users/:id	Roles: admin, super_admin	Users
POST /api/votes/rate	Any signed-in user	Votes
POST /api/votes/upvote	Any signed-in user	Votes
POST /api/votes/downvote	Any signed-in user	Votes
POST /api/votes/community	Any signed-in user	Votes

Method and path	Guard	Module
GET /api/votes/community-stats	Any signed-in user	Votes
GET /api/votes/poll-all	Any signed-in user	Votes
GET /api/votes/stats	Any signed-in user	Votes
GET /api/votes/board	Any signed-in user	Votes
GET /api/health, GET /api/ready	Open (operational probes)	Operations

13. Appendix C - Configuration reference

Every value below is read from the environment at start-up. They are listed here because several assurance claims in this document - capacity, lockout behaviour, session lifetime, upload limits - are only true for a given configuration, and because the extensibility cases assert that each of these is configuration rather than code.

Setting	Default	What it governs
JWT_SECRET	none - refused in production	Signing key for session tokens. A production server will not start with the example value.
JWT_EXPIRES_IN	28800 (8 hours)	Session lifetime. Shorter is safer; the live user check limits the damage either way.
MIN_PASSWORD_LENGTH	12	Minimum accepted password length.
GLOBAL_RATE_LIMIT	300 per minute per IP	Coarse abuse cap across the API. Raise it for a large office behind one address.
AUTH_RATE_LIMIT	30 per 15 minutes per IP	Failed-authentication budget; successful logins do not count against it.
DB_POOL_SIZE	10 per tenant	Database connections per tenant schema. Multiply by (tenants + 1) and keep below the server limit.
MAX_FILE_MB	10	Attachment size limit, enforced with a clean error rather than a dropped connection.
POINTS_SUBMIT / POINTS_APPROVED / POINTS_IMPLEMENTED	10 / 25 / 65	Recognition scheme values.
CORS_ORIGIN	localhost dev server	Origins allowed to call the API from a browser. Required in production.
FRONTEND_BASE_URL	localhost dev server	Base for emailed links such as password resets. Must be HTTPS in production.
FORCE_HTTPS	on outside development	Redirects plaintext requests and enables HSTS.
QCMS_BASE_URL	local development endpoint	Default endpoint for the implementation system; each organisation may override it from its own admin screen.
AI_PROVIDER / OPENAI_API_KEY / GEMINI_API_KEY	blank	Optional scoring provider. With none configured the built-in heuristic is used and the feature still works.
LOG_TO_FILE / LOG_DIR	on in production	Durable daily log files, with errors also written separately.
BACKUP_DIR / BACKUP_KEEP	backend/backups, 14	Backup destination and retention.

14. Appendix D - Test data and accounts

Every account and every row used by the run is created by the run. Nothing depends on a database that happens to be lying around, which is what makes the results reproducible on any machine.

Organisation A ("orga")	Default tenant. Seeded with one administrator and one employee; receives the bulk load of 5,000 ideas and 300 users during the performance module.
Organisation B ("orgb")	Second tenant. Exists so that every isolation case has a real neighbour to fail against, and so the performance module can show one organisation is unaffected by the other's volume.
Organisations created during the run	"Acme Foods" and "Growth Co" are provisioned by the product itself, through the platform administration API, and are used to prove runtime onboarding and schema parity. Their databases are dropped afterwards.
Throwaway accounts	Purpose-built users are seeded for the cases that must deactivate, delete, expire or lock an account, so no case damages another case's fixture.
Ideas	A handful of hand-written ideas carry the functional cases; the 5,000 generated ideas exist only to give the performance cases a realistic table to work against.
Attachments	A genuine one-pixel PNG is used for the valid upload; a text buffer with a forbidden extension and an oversized buffer are used for the rejection cases.
External systems	Local stand-in servers reproduce the documented QCMS responses (201, 409, 401), a refused connection and a server that never answers.

15. Appendix E - Requirement traceability

The commitments a buyer is most likely to write into a contract, and the cases that demonstrate each one.

Commitment	Demonstrated by
One organisation can never see another organisation's data	TC-SEC-001, TC-SEC-002, TC-EXP-003, TC-SUP-002, TC-SAFE-023, TC-SAFE-030
Vendor staff cannot read customer content	TC-SAFE-030, TC-PLAT-008, TC-SUP-003
Offboarding ends access immediately	TC-SAFE-008, TC-SAFE-009, TC-AUTH-008
A password change invalidates older sessions	TC-SAFE-007
Privileges cannot be self-granted	TC-SAFE-004, TC-SAFE-005, TC-SAFE-006, TC-USER-004
Credentials and secrets are never returned to a client	TC-SAFE-011, TC-BRND-003, TC-QCMS-002, TC-SAFE-037
Anonymous submissions stay anonymous	TC-SAFE-027, TC-SAFE-028, TC-SAFE-029
Uploaded files are private, validated and non-executable	TC-SAFE-017 to TC-SAFE-026
Brute-force attempts are throttled and cannot be evaded	TC-AUTH-011, TC-SAFE-033, TC-SAFE-034, TC-SCLH-011
A decision is recorded exactly once, with an audit trail	TC-RVW-005, TC-REL-010, TC-DATA-009
No user action is lost when a dependency fails	TC-REL-012, TC-REL-013, TC-REL-014, TC-REL-015
The service can be scaled out behind a load balancer	TC-SCLH-001 to TC-SCLH-016
Performance does not degrade as the organisation grows	TC-SCLV-002 to TC-SCLV-021
Schema changes are applied exactly once and are reversible in a rebuild	TC-DATA-001 to TC-DATA-005

Commitment	Demonstrated by
Data can be backed up and restored	TC-DATA-015, TC-DATA-016
New organisations can be onboarded without a deployment	TC-PLAT-001, TC-SCLH-012, TC-FUT-005
The product can be extended without a rewrite	TC-FUT-001 to TC-FUT-018
Operations can monitor, drain and diagnose the service	TC-OPS-001 to TC-OPS-008

16. Appendix F - Glossary

Tenant	One customer organisation, with its own database schema and its own users.
Slug	The short organisation code (for example "acme") that identifies a tenant in the registry and inside a session token.
Registry	The master database holding the tenant list, platform administrators, login-attempt counters and the migration ledger.
Session token	A signed token carrying who is claiming to be logged in. Role and status are re-read from the database on every request; the token is not trusted for either.
Escalation	An approval that moves an idea up the reporting line to a higher reviewer instead of closing it.
Workflow trail	The per-idea audit list of who did what, and when.
Idempotent	Safe to repeat: doing it twice leaves the same result as doing it once.
Migration ledger	The table recording which schema change has been applied to which database, so migrations are never applied twice and never skipped.
Liveness / readiness	Liveness says the process is up; readiness says it can reach its database and should receive traffic.
Horizontal scaling	Adding more application servers behind a load balancer.
Vertical scaling	Handling more data and more load on the same server.

17. Appendix G - The assertion suite

Alongside the case runner, the project carries an assertion suite that runs on every push through continuous integration. It covers the same application in a different style - hard assertions that fail the build rather than a document - and includes the mapper unit tests and the full push flow against a stand-in QCMS server. Its tests are listed below, read from the source at build time.

#	Test
1	wrong password is 401 and counts toward logout
2	nonexistent email gets the same 401 as a wrong password (no enumeration)
3	5 failures lock the account for 15 minutes
4	a tampered JWT signature is rejected
5	a token with a stale pwd_ts is rejected (password change revokes sessions)
6	a ticket raised in org A is invisible to org B - list and direct read
7	an employee sees only their own tickets; their org admin sees the org
8	IFQM internal notes never reach any tenant reader
9	tenants may close their ticket but never set IFQM triage statuses
10	any user reads branding; only admins write it
11	logo upload validates the actual bytes, not the filename
12	saving unrelated tenant settings never wipes a stored SMTP password
13	platform console never returns smtp_pass, and empty writes preserve it
14	platform tenant views expose no employee PII - only admin contacts
15	the old org-chart endpoint stays dead
16	tenant admins cannot reach platform endpoints
17	suspending a tenant blocks its logins; reactivating restores them
18	the default org cannot be suspended; deletion requires the org code
19	admin password reset is scoped to admins only
20	platform admin account guards hold
21	notification polling as a platform admin returns empty, not 500
22	an employee can attach a document to the Support Required section
23	the single-idea Closure Summary PDF honours the review hierarchy and tenant boundary
24	a user signs in with just their email - no organisation code needed
25	a user signs in with their registered phone number
26	an unknown email is refused generically, exactly like a wrong password
27	an idea can credit more than two co-suggesters
28	a document can be attached to the Benefits Expected section
29	the dashboard reports monthly submission activity
30	QCMS payload maps our idea onto the documented field shape
31	QCMS integration: approved-ideas list, key masking, and the push flow

#	Test
32	the QCMS base URL can be overridden per tenant from the admin dashboard
33	a QCMS duplicate-key leak (HTTP 500) is treated as a duplicate, not a failure

33 tests, run with "npm test" from the backend directory. All passed in this cycle.

18. Conclusion

The platform passed 228 of 228 cases in the run recorded here, across functional behaviour, safety, reliability, both directions of scalability, data integrity, extensibility and operations. The cycle was worth running on its own terms: it found three genuine defects - two of them only reachable under real concurrency, one of them a privacy commitment that was broken in a field nobody had looked at - and each is now fixed and permanently guarded by a case that would have caught it.

The claims that matter are backed by evidence in this document rather than by assertion: one organisation cannot reach another's data because it is in another schema; a departing employee's session dies on their next request because the row is re-read every time; the service runs unchanged behind a load balancer because a second instance was started and proved interchangeable; and screen latency does not follow data volume because the work per request is capped and indexed. The limitations are stated with the same directness in section 9, and the next actions are listed there.

Role	Name	Signature	Date
Prepared by (QA)			
Reviewed by (Engineering)			
Approved by (Product / Quality)			
Customer acceptance			

End of document. 228 cases, 228 passed, 0 failed, run 2026-07-28 17:35:05.